Reading the agent instead of running the simulator: model-internal signals for selective digital-twin validation of large language model agents in process control

Javal Vyas[a,*], [TODO: co-authors][a]

[a] *[TODO: Department], Imperial College London, London, United Kingdom*

Abstract

Large language model (LLM) agents that propose corrective actions for industrial processes are guarded by a digital twin that simulates every proposal before execution. Simulation cost grows with horizon and fidelity, whereas the agent's forward pass, which already computes token probabilities, attention and hidden states, costs a fixed amount. We ask whether these model-internal signals can replace part of the simulation. A calibrated probe on the agent's internals routes each proposal to execution, rejection or simulation, with a Clopper–Pearson guarantee on the failure rate of unchecked executions. In a pre-registered study of five open-weight models and two domains (graph path planning; fault recovery of a continuous stirred-tank reactor), internals alone, with no plant model, removed 11–79% of validator calls in path planning and 12–53% in the reactor, mostly through rejections that were 95–98% precise. In path planning, internals added 11–37 points to observables that do not reuse the validator's logic; in the reactor, a flexible surrogate of the plant absorbed their contribution. Attention on the prompt region

*Corresponding author

Email address: j.vyas24@imperial.ac.uk (Javal Vyas)

Preprint submitted to Engineering Applications of Artificial Intelligence November 2, 2026

that decides the answer explains why: it predicts failures to consult relevant information (AUROC 0.72–0.88), not failures of reasoning over consulted information (0.48–0.72). In closed loop, internals-based probes accepted failing retries, a shift outside the guarantee. Allowing unchecked execution only for first proposals restored safety (internals-only screens: 0–2 unsafe executions per 400 episodes, none in a pre-registered replication) while avoiding 29–65% of validator calls. A screen cannot replace the simulator's feedback, though: an agent that repairs from it lost 3.5 points of recovery.

Keywords: Large language models, Model internals, Selective verification, Digital twin, Fault-tolerant control, Risk certification

1. Introduction

Large language model (LLM) agents are increasingly proposed as supervisory decision makers for engineered systems. They suggest recovery setpoints after a process fault, operating sequences for start-up or changeover, or plans that lower-level controllers then execute [1, 2, 3, 4, 5]. An LLM's proposal can be wrong in ways that are difficult to anticipate from its wording. Such agents are therefore deployed behind a *validator*: before a proposal reaches the plant, a simulator or digital twin [6] rolls the plant forward under the proposed action and checks it against the operating constraints. A rejected proposal is returned to the agent with the validator's feedback, and the agent tries again. This propose–validate–retry loop makes the agent safe by construction: no proposal reaches the plant without having passed a simulation.

The price of this safety is that *every* decision pays for a full simulation. In process engineering the simulation is often the expensive step. A digital-

twin rollout must integrate the plant model over the recovery horizon; its cost grows with the horizon, the fidelity of the model (lumped versus distributed parameters, detailed thermodynamics, coupled units) and the number of disturbance scenarios checked, and it can range from seconds to hours per call. The agent's own cost, in contrast, is fixed for a given prompt: one forward pass per generated token. As validators become more faithful, they become the bottleneck of the loop, and the question of *which* proposals actually need simulating becomes an engineering question with direct consequences for the latency and cost of LLM-based supervision.

This paper starts from an observation about that forward pass. To produce a proposal, the LLM computes, and then discards, a large amount of information about its own decision: the probability it assigned to each output token, the attention weights that show which parts of the prompt it consulted while writing each token, and high-dimensional hidden states at every layer. A growing literature shows that such *model-internal* signals carry information about whether the model's output is correct [7, 8, 9, 10, 11, 12, 13]. If they also carry information about whether a proposal will pass the validator, they offer something a simulator cannot: a risk estimate that is available at no simulation cost, before the simulator is invoked, from a signal that does not depend on any model of the plant. The central question of this paper is therefore:

> *Can an LLM agent's internal signals replace part of the simulation-based verification of its proposals, and if so, how much, at what risk, under which conditions, and why?*

We decompose this question into four research questions, each answered

by a pre-registered analysis.

RQ1 (substitution) How many validator calls can model-internal signals alone eliminate, with a certified bound on the failure rate of the proposals executed without validation, when no model of the plant is used?

RQ2 (complementarity) Do internal signals add to *observable* features of the task state and the proposed action, which are the natural engineering baseline?

RQ3 (mechanism) When do internal signals help, and can the reason be measured? We test whether failures that arise from *not consulting* the relevant prompt information (retrieval failures) are detectable from attention, whereas failures that arise from *reasoning* incorrectly over correctly consulted information are not.

RQ4 (deployment) Do internals-based screens remain safe inside the closed propose–validate–retry loop, and under which validator costs do they reduce the total compute?

To answer these questions we turn the validator into a three-way router (Fig. 1). A calibrated probe reads signals from the agent's forward pass and estimates the probability that the proposal will fail validation. A confident proposal is executed without simulation (ACCEPT), a clearly failing proposal is returned to the agent without simulation (REJECT), and only the uncertain remainder is simulated (VALIDATE). The accept threshold is chosen so that a one-sided Clopper–Pearson bound [14] on the failure rate among unchecked executions stays below a tolerance, and that bound is then certified once on

data that played no role in building the screen. We compare five families of signals: token confidence, attention mass by prompt region, hidden states, a targeted *region-based attention grounding* signal that measures whether the model attended to the specific part of the prompt that decides each answer token, and observable features of the task and action. Combinations of these families are compared as well.

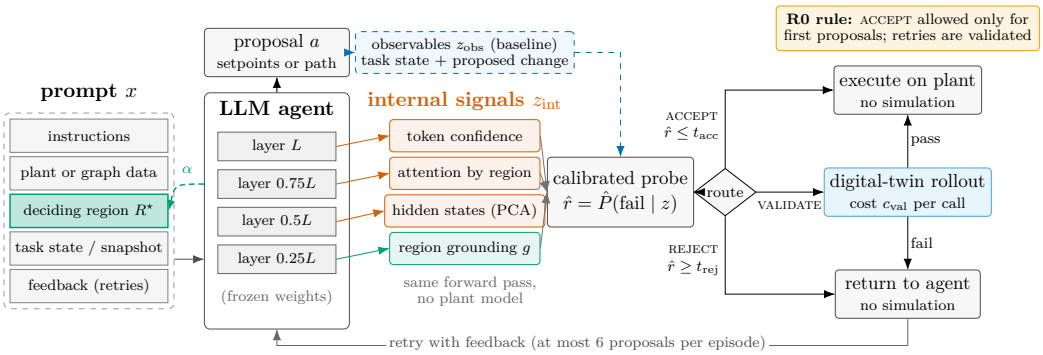


Figure 1: Selective validation driven by the agent's own internals. The forward pass that produces a proposal a from a prompt x also yields token confidence, attention by prompt region, hidden states at four relative depths, and region-based attention grounding g (the share of attention α that falls on the prompt region $R^\star$ that decides each answer token). A calibrated probe maps these signals, optionally together with observable features of the task and proposal, to a failure risk $\hat{r}$. Proposals are executed without simulation (ACCEPT), returned to the agent without simulation (REJECT), or simulated by the digital twin (VALIDATE). Both ACCEPT and REJECT avoid a validator call. Under the first-proposal-only rule (R0, Section 7), retries are never accepted unchecked.

We study two domains chosen to differ in the type of error the agent makes. In *graph path planning* the agent must return a path through a directed graph listed in the prompt, and the validator is exact: a step is legal

only if the target appears in the source node's adjacency line. Errors are typically retrieval errors, in which the model writes a step without consulting the line that decides it. In *fault recovery of a continuous stirred-tank reactor (CSTR)* [5] the agent must propose three controller setpoints after a process fault, and a digital twin validates each proposal by simulation. Here the relevant measurements and physics are short and salient, and errors are typically reasoning errors in the direction or magnitude of the correction. Five open-weight models from three families, Qwen2.5 1.5B, 3B and 7B [15], Llama-3.2-3B [16] and SmolLM2-1.7B [17], generate the proposals. All hypotheses, decision rules and amendments were pre-registered before the corresponding data were examined [18]. Test and certification partitions were evaluated once, and the closed loop was run on 400 fresh episodes per model.

The contributions of this work are as follows.

1. **A formulation of simulator replacement by model internals as certified three-way routing** (Section 3). We state the guarantee that the bound-based accept rule provides, the exchangeability assumption under which it holds, and an exact, linear cost model. The cost model yields the break-even validator cost above which screening saves compute.
2. **Evidence that the agent's internals alone, with no plant model, remove a substantial share of verification** (RQ1, Section 6.1). Internals-only screens skipped 11–79% of validator calls in path planning, with certified accepts for four of five models. In the CSTR they skipped 12–53% of calls, almost entirely by rejecting proposals, 95–98%

of which indeed failed.

3. **A fair test of complementarity** (RQ2, Section 6.2). We compare against observable baselines of increasing strength: a flexible probe, and features that do not reuse the validator's own logic. In path planning the internals still add for four of five models under a flexible probe, and add 11–37 points for all five against validator-free observables. In the CSTR, a flexible observable surrogate of the plant absorbs their contribution.
4. **A mechanistic account of when internals help** (RQ3, Section 6.3). Low attention on the deciding prompt region predicts failure in path planning (AUROC 0.72–0.88). It predicts weakly or not at all in the CSTR (0.48–0.72), where a pre-registered test shows that failing proposals attend to the relevant physics *more*, not less.
5. **Closed-loop evidence, a failure mode and a deployment rule** (RQ4, Section 7). In 2000 closed-loop episodes, trained screens executed far fewer failing actions than random skipping at matched rates. Probes that use internals accepted failing *retries* for two models. This is a distribution shift that lies outside the certified guarantee. Restricting unchecked execution to first proposals restored the guarantee's premise: it reduced failing executions to 0–2 per 400 episodes for the internals-only screens while still avoiding 29–65% of validator calls.

The remainder of the paper is organised as follows. Section 2 reviews related work. Section 3 formalises the routing problem, the guarantee and the cost model. Section 4 defines the internal and observable signals and the probes. Section 5 describes the domains, models, data and statistical

protocol. Section 6 answers RQ1–RQ3 offline, and Section 7 answers RQ4 in closed loop. Section 8 discusses implications, gives a deployment guide and examines threats to validity. Section 9 concludes.

2. Related work

LLM agents for process systems and their safeguards. Generative models are being adopted across process systems engineering, from flowsheet synthesis to operator support [3]. Agentic designs interleave reasoning with actions and revise their proposals from environment feedback [1, 2]. In process control, LLM agents have been used for supervisory fault recovery, with proposals checked against plant knowledge and digital-twin rollouts before execution [4, 5]. Learning-based control has a long tradition of guarding a learned policy with a model-based safety layer, such as predictive safety filters [19] and shields for reinforcement learning [20]. These safeguards assume that every action is checked. The present work keeps the safeguard and asks which checks can be skipped, using information that the agent produces anyway.

Surrogates and multi-fidelity evaluation. When simulation is expensive, engineering practice replaces part of it with cheaper surrogates of the simulator [21]. An observable screen that predicts the validator's verdict from plant measurements and the proposed action is such a surrogate: it learns the plant's response. A screen built on the agent's internals is of a different kind. It does not model the plant at all, but the agent's own propensity to err. It therefore needs no plant data beyond the validator's labels. Comparing the two kinds of screen, and their combination, is a central part of this study (RQ1, RQ2).

Selective prediction, deferral and risk control. Selective classification lets a model abstain when its risk is high, trading coverage for accuracy [22]; selective question answering applies the idea to language models [23]. Learning-to-defer methods decide when to pass a case to an expert [24, 25]. Distribution-free methods calibrate such decisions so that a risk level is guaranteed with high probability [26, 27, 28]. Our accept rule belongs to this family: an upper confidence bound on the failure rate of accepted proposals is kept below a tolerance, and the bound is verified once on independent data with an exact binomial interval [14]. Two features distinguish our setting. First, the decision has three outcomes, and both non-deferral outcomes avoid the expensive expert, here the simulator. Second, the screen operates inside a loop in which a rejection produces a new proposal from a changed prompt. That loop raises a distribution-shift problem that static selective prediction does not face (Section 7).

Model internals as signals of correctness. Token probabilities carry information about answer correctness [7], and verbalised confidence is a weaker substitute [29]. Linear probes on hidden states [30, 31] detect falsehoods, latent knowledge and hallucinations [8, 9, 10, 11], and the representations of a model can encode more about the correctness of its answer than its output reveals [12]. Semantic entropy detects confabulations from sampled answers [32, 33], and probes can approximate it from a single forward pass [34]. These studies ask whether an answer is likely wrong. We ask a decision question: how many simulator calls such signals let an agent skip at a certified risk, whether they add to strong observable baselines, and whether they stay safe in closed loop.

Attention as evidence of grounding. Whether attention weights explain model behaviour is debated [35, 36]; we use attention only as a predictive signal and make no explanatory claim. Lookback Lens detects contextual hallucinations from the ratio of attention on the context to attention on generated tokens [13]. Our region-based grounding is a targeted variant of this idea. It measures the share of attention, while each decision-relevant token is written, that falls on the specific prompt region that decides that token: an adjacency line in path planning, and pre-registered measurements and knowledge-graph relations in the CSTR. Our results delimit when such a signal is useful. It works when failures are failures to retrieve the relevant information. It does not work when they are failures of reasoning over information that was retrieved.

Table 1 summarises how the present study relates to these lines of work.

Table 1: Positioning of this study. ✓: addressed; (✓): partially addressed; –: not addressed.

	Plant-model-free risk signal	Certified risk on accepts	Three-way routing	Closed loop with retries
Safety filters, shields [19, 20]	–	✓	–	✓
Simulator surrogates [21]	–	–	–	(✓)
Selective prediction, risk control [22, 27]	(✓)	✓	–	–
LLM internals as correctness signals [8, 11, 13]	✓	–	–	–
LLM agents with validators [1, 4]	–	–	–	✓
This study	✓	✓	✓	✓

3. Problem formulation

This section formalises selective validation as a three-way routing problem. It states the guarantee that the accept rule provides and the assumption on which that guarantee rests. It then derives the exact cost model used to judge whether screening pays off.

3.1. Agent, validator and episode

An *episode* begins when a monitoring layer detects that the process has left its safe operating region, or, in path planning, when a query is posed. In round $k = 0, 1, \ldots, K-1$ the agent receives a prompt x_k and returns a proposal a_k. The first prompt x_0 describes the task state. A later prompt x_k ($k \geq 1$) additionally contains feedback on the previous proposal. The *validator* $V(x, a) \in \{0, 1\}$ returns whether proposal a is acceptable. In the CSTR it simulates the plant from the current state under the proposed setpoints and checks the outcome contract (Section 5). We write $Y = 1 - V$ for the failure indicator. In the conventional loop, which we call *always-validate* (AV), every proposal is validated. A passing proposal is executed and ends the episode. A failing one is returned to the agent with the validator's measured outcome as feedback, until a budget of $K = 6$ proposals is exhausted. If no proposal passes, the current setpoints are held.

While generating a_k, the agent's forward pass produces an internal record h_k: output-token distributions, attention weights and hidden states. A *signal map* ψ turns the available information into a feature vector $z = \psi(x, a, h)$. The signal families are defined in Section 4. Some use only h (internal signals), some use only (x, a) (observable signals), and some use both.

3.2. Three-way routing

A *screen* consists of a probe f, which maps z to an estimated failure probability $\hat{r} = f(z) \in [0, 1]$, and two thresholds $t_{\text{acc}} < t_{\text{rej}}$:

$$d(\hat{r}) = \begin{cases} \textsc{accept} & \hat{r} \le t_{\text{acc}} \quad \text{(execute without validation)}, \\ \textsc{reject} & \hat{r} \ge t_{\text{rej}} \quad \text{(return to the agent without validation)}, \\ \textsc{validate} & \text{otherwise.} \end{cases} \tag{1}$$

Both ACCEPT and REJECT avoid a validator call. The two errors differ in kind. An ACCEPT of a failing proposal executes an unsafe action on the plant; this is the safety-relevant error. A REJECT of a passing proposal costs a regeneration and may cost the episode its recovery if the budget runs out; this is an efficiency error. The design therefore treats the two thresholds asymmetrically: t_{acc} carries a statistical guarantee, while t_{rej} is set for high precision.

Definition 1 (Offline operating characteristics)**.** *For a screen applied to proposals drawn from a distribution P, the* skip rate *is $S = P\{d \in \{\textsc{accept}, \textsc{reject}\}\}$, the* unchecked failure rate *is $\rho_A = P\{Y = 1 \mid d = \textsc{accept}\}$, and the* rejection precision *is $\pi_R = P\{Y = 1 \mid d = \textsc{reject}\}$.*

The design goal is to maximise S subject to $\rho_A \le X$ with high confidence, for a tolerance X set by the operator, and subject to a high rejection precision. We use $X = 10\%$ as the primary tolerance and $X = 5\%$ as a secondary one. The validator calls skipped, S, are the primary endpoint of the study. We use this rather than the area under the ROC curve (AUROC) because S

measures what an engineer saves at a fixed risk, whereas AUROC averages over operating points that would never be deployed.

3.3. Choosing the thresholds

Let $\mathcal{D} = \{(\hat{r}_i, Y_i)\}$ be development proposals that were not used to fit the probe. For a candidate threshold t let $m(t) = |\{i : \hat{r}_i \le t\}|$ be the number of proposals it would accept and $k(t)$ the number of failures among them. Let

$$U_\alpha(k, m) = \begin{cases} \mathrm{Beta}^{-1}(1-\alpha;\ k+1,\ m-k) & k < m, \\ 1 & k = m, \end{cases} \tag{2}$$

be the one-sided Clopper–Pearson upper confidence bound at level $1-\alpha$ on a binomial proportion with k successes in m trials [14]; $U_\alpha(0,0) = 1$. The accept threshold is the largest threshold whose bound does not exceed the tolerance,

$$t_{\mathrm{acc}} = \max\{t :\ U_{\delta_{\mathrm{sel}}}(k(t), m(t)) \le X\}, \qquad \delta_{\mathrm{sel}} = 0.05, \tag{3}$$

and the reject threshold is the smallest threshold whose rejected development set is at least 95% failures,

$$t_{\mathrm{rej}} = \min\Big\{t :\ \tfrac{1}{|\{i:\hat{r}_i \ge t\}|} \textstyle\sum_{i:\hat{r}_i \ge t} Y_i \ge 0.95\Big\}. \tag{4}$$

If no threshold satisfies Eq. (3), the screen accepts nothing. Fig. 2 illustrates why the bound matters. A rule that sets t_{acc} where the *point estimate* $k(t)/m(t)$ reaches X selects a larger threshold. The failure rate of the set it selects is then biased upward on new data, because the selection itself favours thresholds whose development failure rate happened to be low. In an earlier pre-registered stage of this study the point rule did prove too loose on

certification data. The bound-based rule replaced it by a dated amendment, made before any affected test data were analysed.

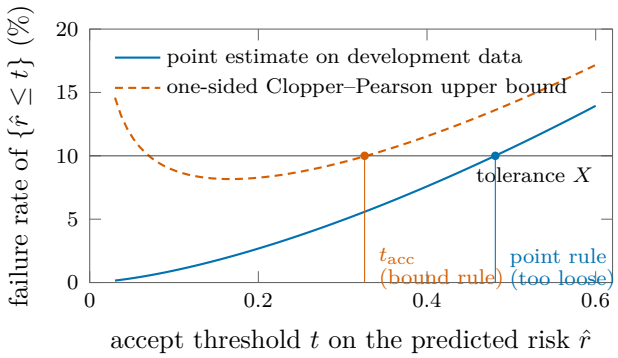


Figure 2: Schematic of the bound-based accept rule (illustrative curves, not data). For each candidate threshold t, the failure rate of the development proposals with $\hat{r} \le t$ (solid) and its one-sided Clopper–Pearson upper bound (dashed) are computed. The bound is large for small t, because few proposals are accepted, and approaches the point estimate as more are accepted. The accept threshold t_{acc} is the largest t at which the bound stays below the tolerance X. The point-estimate rule picks a larger threshold and, after selection, exceeds X on new data.

3.4. What the certificate guarantees

After f, t_{acc} and t_{rej} are frozen, each screen is evaluated once on a certification set $\mathcal{C}$ that played no role in building it. Let m be the number of certification proposals it accepts and k the number of failures among them.

Proposition 1 (Certified unchecked failure rate)**.** *Let the screen be fixed independently of $\mathcal{C}$, and let the proposals in $\mathcal{C}$ and the future proposals on which the screen is deployed be independent draws from the same distribution P. Then, for the deployed unchecked failure rate $\rho_A = P\{Y = 1 \mid \hat{r} \le t_{\mathrm{acc}}\}$,*

$$\Pr\{\rho_A \le U_{\delta/M}(k, m)\} \;\ge\; 1 - \delta/M,$$

and the statements for M screens certified on the same set hold simultaneously with probability at least $1 - \delta$.

Proof. Because the screen does not depend on $\mathcal{C}$, the accept decisions are a fixed function of each certification proposal's features. Conditional on the set of accepted proposals, their failure indicators are therefore independent Bernoulli variables with mean ρ_A, so $k \mid m \sim \mathrm{Bin}(m, \rho_A)$. The Clopper–Pearson bound covers a binomial proportion with probability at least $1-\delta/M$ for every m [14]. The same coverage therefore holds unconditionally. The simultaneous statement follows from the union (Bonferroni) bound. □

We call a screen *certified* at tolerance X if $U_{\delta/M}(k, m) \leq X$ with $\delta = 0.05$ and M the number of signal sets compared on that certification set ($M = 10$ in path planning, $M = 11$ in the CSTR). Proposition 1 has two practical consequences.

A minimum accept-set size. With no failures among the accepts, $U_\alpha(0, m) = 1 - \alpha^{1/m}$, so certification at tolerance X requires

$$m \;\geq\; m_{\min} = \left\lceil \frac{\ln \alpha}{\ln(1 - X)} \right\rceil . \tag{5}$$

For $X = 10\%$ and $\alpha = 0.05/11$, $m_{\min} = 52$; any failure among the accepts raises the requirement. A certification set of 500 episodes in which a screen accepts 10% of proposals therefore cannot certify it, however safe the accepts are. We stated this power limit for the CSTR in advance.

An exchangeability premise. The guarantee covers proposals drawn from the certification distribution and nothing else. This becomes decisive in closed loop, where the agent also produces *retries*.

Remark 1 (Retries are outside the guarantee). The certification sets contain first proposals. A retry a_k, $k \geq 1$, differs from them in two ways. Its prompt x_k contains feedback that a first prompt never contains, which changes the model's internal state. And the retry is drawn only in episodes whose previous proposal was not executed, which selects harder episodes. Proposition 1 therefore gives no bound on ρ_A for retries. The *first-proposal-only rule* (R0) replaces ACCEPT by VALIDATE whenever $k \geq 1$, so that every unchecked execution is a first proposal, the population the certificate covers. Unchecked REJECT remains allowed on retries, since it cannot execute an unsafe action.

3.5. Cost model and break-even validator cost

Let a policy p (always-validate, or a screen with or without R0) be run on an episode. Its compute is

$$C_p(c_{\text{val}}) = \underbrace{\sum_k \big(c_{\text{gen},k} + c_{\text{feat},k}\big)}_{G_p} + N_p\, c_{\text{val}}, \tag{6}$$

where $c_{\text{gen},k}$ is the generation time of proposal k, $c_{\text{feat},k}$ the time of any extra pass needed to compute features (zero for observables), N_p the number of validator calls and c_{val} the cost of one call. In our closed loop the plant is paused while the agent and the validator run, decoding is greedy, and the validator is deterministic. The sequence of decisions, and with it G_p and N_p, is therefore independent of c_{val}. The expected compute $\bar{C}_p(c_{\text{val}}) = \bar{G}_p + \bar{N}_p\, c_{\text{val}}$ is then exactly linear in the validator cost. Logged decisions can be re-costed for any validator, faster or slower than the one measured. Comparing a screen p with always-validate gives the following.

- If $\bar{N}_p < \bar{N}_{\mathrm{AV}}$, the screen saves compute if and only if

$$c_{\mathrm{val}} > c_p^* = \frac{\bar{G}_p - \bar{G}_{\mathrm{AV}}}{\bar{N}_{\mathrm{AV}} - \bar{N}_p}, \tag{7}$$

  and it saves at any validator cost if $\bar{G}_p \leq \bar{G}_{\mathrm{AV}}$.

- The relative saving $1 - \bar{C}_p/\bar{C}_{\mathrm{AV}}$ increases monotonically with c_{val} when $c_p^* \geq 0$ and tends to $1 - \bar{N}_p/\bar{N}_{\mathrm{AV}}$, the fraction of validator calls avoided, as $c_{\mathrm{val}} \to \infty$.

The numerator of Eq. (7) explains when screening is cheap. An ACCEPT of a passing proposal saves a call and costs nothing, since always-validate would also have executed it. A REJECT of a failing proposal saves a call and also costs nothing in expectation, since always-validate would also have asked for a new proposal. A REJECT of a *passing* proposal, in contrast, forces generations that always-validate would not have needed, and feature passes add a fixed overhead. Rejection precision therefore governs the break-even cost, and accept safety governs the risk.

Algorithm 1 summarises one closed-loop episode under a screen.

4. Signals and probes

All internal signals are read from the forward pass that produced the proposal. The stored answer is re-run once with teacher forcing to extract attention and hidden states, and the agreement between the teacher-forced argmax and the generated token is checked. Agreement was 96–99.9% for the bfloat16 models, except Qwen2.5-1.5B in path planning (92.8%). It was lower for the 4-bit Qwen2.5-7B (84% in path planning, 91% in the CSTR),

Algorithm 1 One closed-loop episode with a screen (optional R0 rule)

Require: snapshot s, agent, probe f, thresholds $t_{\text{acc}}, t_{\text{rej}}$, validator V, budget $K = 6$

$x_0 \leftarrow \text{prompt}(s)$
for $k = 0, \ldots, K-1$ **do**
  $(a_k, h_k) \leftarrow \text{agent}(x_k)$ ▷ proposal and internals
  if a_k repeats an earlier proposal **then** reuse its verdict (no new call)
  end if
  $\hat{r} \leftarrow f(\psi(x_k, a_k, h_k))$
  if $\hat{r} \leq t_{\text{acc}}$ **and** ($k = 0$ **or** R0 is off) **then**
    execute a_k; **return** ▷ ACCEPT
  else if $\hat{r} \geq t_{\text{rej}}$ **then**
    $x_{k+1} \leftarrow \text{prompt}(s, \text{rejection notice})$ ▷ REJECT: no metrics
  else
    $v \leftarrow V(s, a_k)$ ▷ VALIDATE: one simulator call
    if $v = 1$ **then**
      execute a_k; **return**
    end if
    $x_{k+1} \leftarrow \text{prompt}(s, \text{measured outcome of } a_k)$
  end if
end for
hold the current setpoints ▷ budget exhausted: unresolved

because quantised kernels make the two passes diverge more often. Signals are taken at four relative depths $\mathcal{L} = \{0.25, 0.5, 0.75, 1.0\}$ of the network, where depth 1.0 is the last layer. Let $\mathcal{T}$ denote the answer tokens and $\mathcal{P}$ the prompt tokens. Let $\alpha_{j,i}^{(\ell,\eta)}$ be the attention weight of head η in layer ℓ from the position that predicts answer token j to prompt token i, and let $\bar{\alpha}_{j,i}^{(\ell)}$ be its mean over heads.

4.1. Internal signals

Token confidence. For answer token j with predictive distribution p_j and selected token y_j, we compute the log-probability $\log p_j(y_j)$, the entropy $-\sum_v p_j(v) \log p_j(v)$ and the margin between the two most probable tokens. The features are their mean, minimum and maximum over all answer tokens and over the numeric tokens. This is the cheapest internal signal and needs no extra pass.

Attention by prompt region. The prompt is partitioned into coarse regions R: system instructions, plant knowledge or graph description, task state, and user instructions. The mass that answer token j places on region R in layer ℓ is

$$A_j^{(\ell)}(R) = \frac{\sum_{i \in R} \bar{\alpha}_{j,i}^{(\ell)}}{\sum_{i \in \mathcal{P}} \bar{\alpha}_{j,i}^{(\ell)}}, \tag{8}$$

averaged over answer tokens. Head-maximum variants and the attention entropy are included as well. The signal asks *where in the prompt* the model looked while answering.

Hidden states. The residual-stream vectors at the last prompt token and their mean over answer tokens, at each depth in $\mathcal{L}$, are standardised. They

are reduced to 64 principal components fitted on the training partition only. Linear probes on such representations are known to recover correctness information [30, 8, 12].

Region-based attention grounding. Each answer contains *decision units*: a step $u \to v$ of a path, or one of the three setpoints. For unit u with decision tokens $\mathcal{T}_u \subseteq \mathcal{T}$, a *deciding region* $R_u^\star$ is fixed in advance. It is the part of the prompt that contains the information on which the unit's correctness depends. The grounding share of unit u is

$$g_u = \frac{1}{|\mathcal{L}|} \sum_{\ell \in \mathcal{L}} \frac{1}{|\mathcal{T}_u|} \sum_{j \in \mathcal{T}_u} \frac{\sum_{i \in R_u^\star} \bar{\alpha}_{j,i}^{(\ell)}}{\sum_{i \in R_{\text{task}}} \bar{\alpha}_{j,i}^{(\ell)}}, \tag{9}$$

where R_{task} contains all task-content tokens (all adjacency lines in path planning; all system, knowledge-graph and snapshot tokens in the CSTR). Template tokens and previously generated tokens are excluded, so g_u is the fraction of the model's attention to task content that falls on the content that matters. The answer-level features are the minimum $g_{\min} = \min_u g_u$, which was pre-registered as primary, the mean over units, per-layer variants, and, in path planning, the fraction of steps for which $R_u^\star$ is the most-attended line.

The deciding regions encode domain knowledge and were fixed before any grounding value was computed. In path planning a step $u \to v$ is legal if and only if v appears in the adjacency line of u, so $R_u^\star$ is that line. The definition is exact. In the CSTR the deciding regions of each setpoint were taken from the process structure stated in the prompt (Table 2). They are the measurements that indicate the setpoint's effect, the current setpoints, the knowledge-graph statement of the loop that the setpoint drives, and the statement of the mass and energy balances (operator PO5). Figure 3

illustrates the intended reading of the signal and the two failure types it is meant to separate.

Table 2: Pre-registered deciding regions $R^\star$ per CSTR setpoint (prompt v3.1). Snapshot fields are single lines of the prompt's plant snapshot; knowledge-graph (KG) statements are complete Turtle statements that mention the operator.

Setpoint	Deciding regions
Feed flow $F_{\text{in,sp}}$	u_{cool}, T_{meas}, current setpoints, KG PO8 (flow loop), KG PO5 (balances)
Level L_{sp}	L_{meas}, u_{pump}, current setpoints, KG PO6 (level loop), KG PO5
Temperature T_{sp}	T_{meas}, u_{cool}, current setpoints, KG PO7 (temperature loop), KG PO5

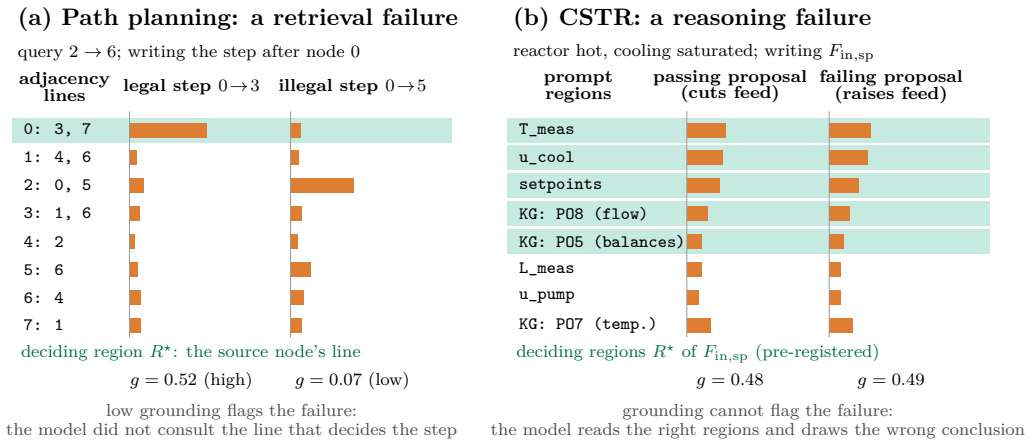


Figure 3: Region-based attention grounding (schematic; bar lengths are illustrative). Each bar is the share of the model's attention to task content that falls on a prompt region while the model writes one decision. (a) In path planning the step after node 0 is legal only if its target appears in line "0: 3, 7". A legal step attends to that line (high g). An illegal step attends elsewhere, here to line "2: 0, 5", which contains the wrongly chosen node (low g). Low grounding flags the *retrieval* failure. (b) In the CSTR, a proposal that cuts the feed and one that raises it while cooling is saturated both attend to the deciding regions. The failing proposal is a *reasoning* failure, and grounding cannot distinguish it.

4.2. Observable signals (baselines)

The engineering baseline uses only information that is visible without opening the model.

- **Path planning:** the numbers of nodes and edges, the edge density, the length of the shortest path (computed by breadth-first search on the prompt's graph), the length of the proposed path, and indicators of whether the path has two nodes, repeats a node or has the correct endpoints. Two of these features use part of the validator's logic (a shortest-path computation and an endpoint check). This makes the observable baseline deliberately strong.

- **CSTR:** the measured temperature and level, the three actuator commands, the anomaly ratio, the control zone, the alarm flags, and the proposed setpoint changes (absolute and, for the feed, relative). A *plant-readings-only* variant omits the action features. Hidden plant parameters, the fault family, its severity and onset time are never features.

The observable CSTR screen is in effect a learned surrogate of the digital twin: it learns how the plant responds to a change from a state. The internals-only screen is not. It sees no plant reading and no proposed setpoint, only the agent's forward pass.

4.3. Signal sets and probes

We compare ten signal sets in path planning and eleven in the CSTR, each fitted as a single probe: observables; plant readings only (CSTR); token

confidence; attention by region; region grounding; hidden states; *all internals* (token confidence, attention and hidden states); all internals with grounding, called *internals only* below; observables with grounding; observables with all internals; and observables with all internals and grounding, called *combined*. Every probe is an ℓ_2-regularised logistic regression fitted on the training partition, with median imputation and standardisation fitted on the same partition, and Platt-calibrated [37] on a separate calibration partition. Logistic probes were chosen deliberately. They are cheap, they are standard for probing [30, 31], and a fixed model class keeps the comparison between signal sets fair. Any advantage of the internal signals must come from the signals, not from a more flexible probe.

5. Experimental design

The two domains were chosen to differ in the *type* of error the agent makes, not only in difficulty. This contrast is what makes RQ3 testable. Path planning has an exact, microsecond validator, so it isolates the quality of the screens. The CSTR has a simulation-based validator, so it measures safety and compute in an engineering setting.

5.1. Domain 1: graph path planning with an exact validator

Each instance is a directed graph with 5, 10, 15 or 20 nodes. It is given in the prompt as one adjacency line per node ("`u: v1, v2`"), together with a start node, a goal node and one worked example on a different graph. The agent returns the path as a JSON list. The validator checks that the path starts and ends at the right nodes and that every step $u \to v$ is an edge, that is, that v appears in the adjacency line of u. Graphs are disjoint

across all data sets. For each model, probes were developed on 3000 graphs (1500 training, 500 calibration, 500 threshold selection, 500 test). They were evaluated once on a fresh certification set of 3000 new graphs that had not been used in any earlier analysis. A fresh set was used because the original test and certification graphs had been examined in earlier exploratory analyses.

5.2. Domain 2: CSTR fault recovery with a digital-twin validator

The plant is the CSTR case study of **(author?)** [5] (Fig. 4). An exothermic reaction $A \rightarrow B$ takes place in a jacketed vessel. Three PID loops regulate the feed flow (FC), the level, by the outlet pump (LC), and the temperature, by the coolant valve (TC). A monitoring layer triggers the agent when the process leaves its safe operating region. Each episode draws a nominal plant: heat-transfer coefficient, feed and coolant temperatures, inlet concentration, coolant capacity, and nominal level and feed setpoints. It also draws one of four fault families (jacket fouling, outlet-pump degradation, cooling valve stuck partly closed, outlet blockage), a continuous fault severity and an onset time between 1500 and 5000 s. The plant is simulated until the first monitor trigger, and the snapshot at that moment is the agent's task. Episodes that never trigger, or that trigger before the fault onset, are redrawn within the same fault family.

The prompt contains the agent's role and the plant's knowledge graph in Turtle syntax: process operators, controllers, and the mass and energy balance equations. It also contains the snapshot (measurements, actuator commands, alarms and current setpoints), the goal and the admissible setpoint ranges. The prompt is deliberately *neutral*: it states the plant physics

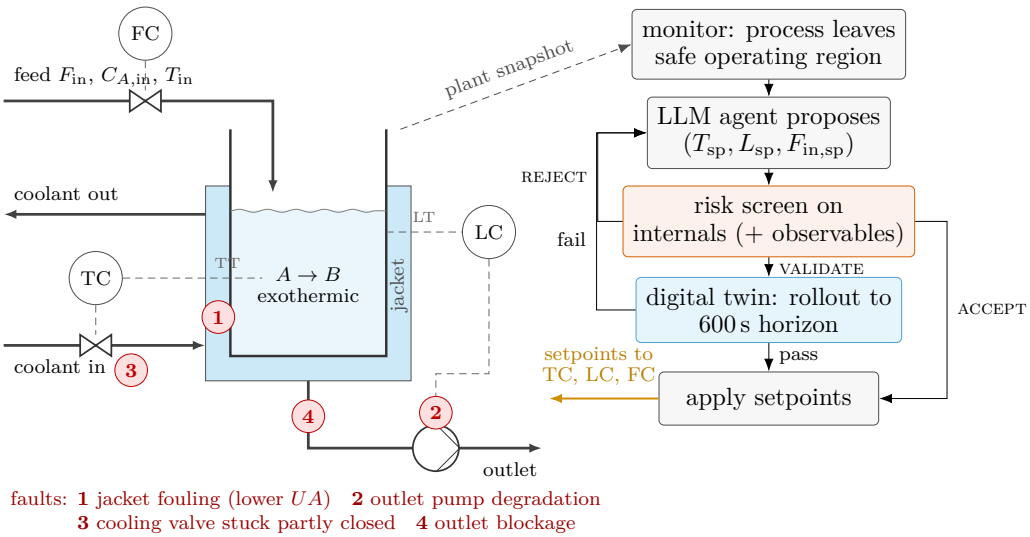


Figure 4: CSTR case study. Left: jacketed reactor with flow (FC), level (LC) and temperature (TC) loops and the four fault locations. Right: supervisory loop. When the monitor detects that the process has left its safe operating region, the LLM agent proposes new setpoints for the three loops. The risk screen accepts the proposal without simulation, rejects it without simulation, or passes it to the digital twin, which simulates the plant forward from the snapshot.

and the acceptance criteria but gives no advice on which setpoint to change, in which direction or by how much. Failures therefore reflect the model's own reasoning rather than the wording of the prompt. The agent returns three setpoints (T_{sp}, L_{sp}, $F_{\mathrm{in,sp}}$) as JSON after a short justification.

The validator simulates the plant forward from the snapshot under the proposed setpoints, with the plant's own future disturbances, to the end of the horizon. A proposal passes if the plant reaches the safe state within 600 s, holds it for 60 consecutive seconds, and remains safe for at least 80% of the remaining run. The safe state means a temperature of at most 313 K, a level between 2 and 11, no overflow, and no actuator saturated while the temperature is away from 310 K. One validation took 8–12 s of wall time on our hardware, with four parallel simulation workers.

Fault severities were calibrated without the LLM so that the reference model's (Qwen2.5-3B) first proposals pass in roughly 30% of episodes. This calibration is a property of the data set, not a target for the agents. It ensures that enough passing proposals exist to train and certify an ACCEPT decision. First-proposal failure rates on the evaluation sets range from 66% to 84% across models (Table 3). The 5000 episodes are partitioned by episode into training (3000), calibration (500), threshold selection (500), test (500) and certification (500), with equal numbers per fault family in each partition (Fig. 5). A further 400 fresh episodes, 100 per fault family and disjoint from all others, are reserved for the closed loop.

5.3. Models and inference

Qwen2.5-1.5B, Qwen2.5-3B and Qwen2.5-7B [15], Llama-3.2-3B [16] and SmolLM2-1.7B [17], all instruction-tuned, were run locally in bfloat16. The

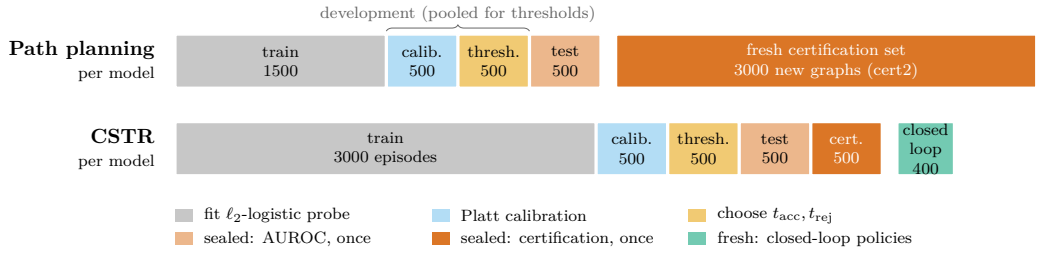


Figure 5: Data partitions per model (widths proportional to size). Each partition has a single role. Thresholds are chosen on the pooled calibration and threshold-selection partitions. The test and certification partitions were sealed until the screens were frozen and were then evaluated once. The closed-loop episodes are fresh.

Table 3: Evaluation sets used for the confirmatory results (one candidate per graph or episode). Path planning: fresh certification set, never used before evaluation. CSTR: sealed test and certification partitions, each evaluated once.

Domain	Model	Set	Eligible	Failure rate
Path planning	Qwen-3B	cert2 (fresh)	2992	0.51
Path planning	Llama-3B	cert2 (fresh)	2995	0.63
Path planning	Qwen-1.5B	cert2 (fresh)	2945	0.68
Path planning	SmolLM-1.7B	cert2 (fresh)	2946	0.83
Path planning	Qwen-7B	cert2 (fresh)	2973	0.49
CSTR	Qwen-1.5B	test_iid	494	0.67
CSTR	Qwen-1.5B	cert	492	0.66
CSTR	Qwen-3B†	test_iid	493	0.71
CSTR	Qwen-3B†	cert	498	0.67
CSTR	Qwen-7B	test_iid	499	0.79
CSTR	Qwen-7B	cert	498	0.80
CSTR	Llama-3B	test_iid	473	0.84
CSTR	Llama-3B	cert	474	0.83
CSTR	SmolLM-1.7B	test_iid	499	0.67
CSTR	SmolLM-1.7B	cert	496	0.66

exception is Qwen2.5-7B, which was run with 4-bit NF4 weights [38] to fit an 8 GB laptop GPU (NVIDIA RTX 4060). Decoding is greedy, so every proposal is reproducible. The knowledge graph was exported once and frozen, so prompts are identical across runs. The models span three families and 1.5 to 7 billion parameters. They were chosen because their weights, attention and hidden states are accessible. API-served models do not expose these signals, a limitation we return to in Section 8.

5.4. Pre-registration, data hygiene and statistics

For each study we froze a written pre-registration before computing the relevant features or outcomes. It fixed the hypotheses, the direction of each predicted effect, the primary features, the decision rules and the analysis [18]. Later changes were recorded as dated amendments made before the affected data were examined: the change of primary endpoint from AUROC to validator calls skipped, the bound-based accept rule, two added signal combinations that isolate grounding, the closed-loop policies, the internals-only closed-loop policy, the first-proposal-only rule, and a pilot gate for SmolLM2 in the CSTR. Test and certification partitions were evaluated once per model with frozen probes and thresholds.

Two analyses are labelled exploratory. The first is the offline CSTR result for Qwen2.5-3B under the bound-based rule: its sealed partitions had been used once under the earlier point rule, before the amendment. The second is the CSTR closed-loop probes of Qwen2.5-3B, which come from that exploratory freeze; the closed-loop episodes themselves are fresh. Where a run failed for technical reasons (GPU memory exhaustion, a crashed simulator worker), it was resumed from its saved records, and no records were

discarded.

The unit of independence is the graph or episode. Differences between screens on the same proposals are reported as paired differences with 95% bootstrap confidence intervals over graphs or episodes (2000 resamples), and a difference is called significant when its interval excludes zero. Closed-loop policies were run on the same 400 episodes and are compared with always-validate by paired bootstrap over episodes. Certification uses the Bonferroni-corrected exact bound of Proposition 1. All numbers in the tables and figures are generated by scripts from the saved results.

6. Results: what the agent's internals can replace, and why

This section answers RQ1–RQ3 on the sealed evaluation sets, with every screen frozen beforehand. Table 4 summarises the answers. Each subsection then opens with its finding and one figure that carries the evidence. All numbers refer to the tolerance $X = 10\%$ on the path-planning fresh certification set and the CSTR certification partition. The complete per-signal tables are in Appendix A.

6.1. RQ1: how much simulation can the agent's internals replace?

Finding. Without any model of the plant, the agent's internals remove 11–79% of simulator calls in path planning, with certified unchecked execution for four of five models. In the CSTR they remove 12–53%, almost entirely by returning proposals that would indeed have failed.

Figure 6 shows, for every model, what the internals-only screen did with each proposal: executed it without simulation, returned it to the agent without simulation, or passed it to the validator. The screen sees only the agent's

Table 4: Summary of findings. Ranges are over the five models; Section 7 covers RQ4.

Question	Path planning (retrieval-heavy)	CSTR fault recovery (physics-heavy)
RQ1: how much simulation do internals alone replace?	11–79% of calls; certified unchecked execution for 4 models	12–53% of calls, almost only by rejection; 95–98% of rejections correct
RQ2: do internals add to observables?	Yes for 4–5 models; against validator-free observables +11 to +37 points	Only with a linear probe (2 models); a flexible plant surrogate absorbs them
RQ3: why?	Low attention on the deciding line predicts failure (AUROC 0.72–0.88)	Models attend to the right physics and still err (AUROC 0.48–0.72)
RQ4: safe and worthwhile in closed loop?	–	Internals-only screen with R0: 0–2 unsafe executions in 400 episodes, 29–65% of calls avoided; saves compute above 3–18 s per call

forward pass: token confidence, attention by region, hidden states and region grounding. It sees no plant reading, no graph computation and no proposed setpoint.

Path planning. The screen skipped 38%, 23%, 11% and 36% of simulations for Qwen2.5-1.5B, Qwen2.5-3B, Qwen2.5-7B and Llama-3.2-3B. For each of the four, the failure rate among the proposals it executed unchecked was certified below 10%: 8 failures among 240 accepts, 21 among 483, 12 among 303 and 13 among 371. The Bonferroni-corrected upper bounds were 6.8–7.8%. For Llama-3.2-3B, the internals produced a certified screen where the observable features did not (upper bound 6.8% against 12.4%). SmolLM2's proposals fail 83% of the time. For it the screen accepted nothing, but it rejected 79% of the proposals, 95% of them correctly.

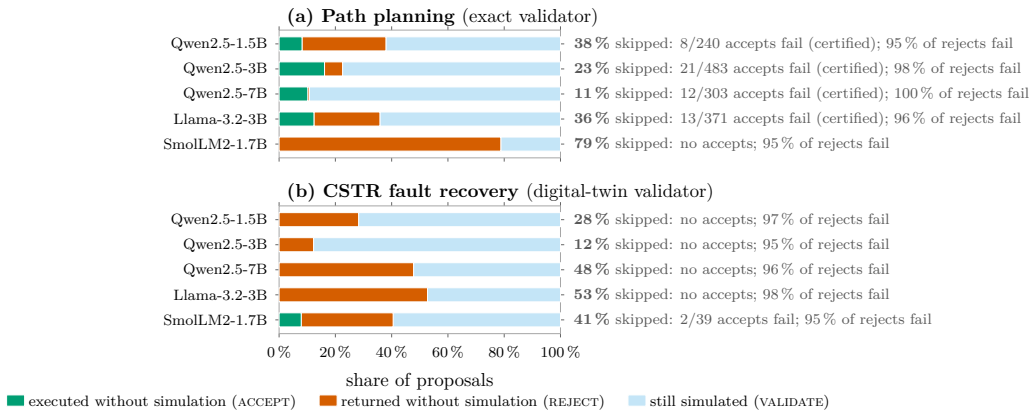


Figure 6: What the agent's internals alone decide, with no model of the plant (internals-only screen, $X = 10\%$). Green: proposals executed without simulation. Orange: proposals returned to the agent without simulation. Light blue: proposals still simulated. The notes give the share of simulations skipped, the failures among the unchecked executions (and whether their rate is certified below 10%), and the share of returned proposals that would indeed have failed. The CSTR Qwen2.5-3B row is exploratory.

CSTR. The screen skipped 28%, 12%, 48%, 53% and 41% of simulations, and the mechanism is different (Fig. 6b). It executed almost nothing unchecked: no proposal for four models, and 39 for SmolLM2, of which 2 failed. What it did was recognise failing proposals. Of the proposals it returned to the agent without simulation, 95–98% would have failed validation. An observable screen on plant readings and the proposed change skipped more calls (41–86%) with similar rejection precision (92–97%), because it can also accept.

Interpretation. The agent's internals carry reliable evidence that a proposal is *bad*. In the retrieval-heavy task they also carry enough evidence that a proposal is *good* to certify unchecked execution. One reading, consistent with RQ3 below, is that internal signals detect the failures that leave a trace in the forward pass, such as misplaced attention, low confidence or an atypical representation. That suffices to reject with high precision. Accepting safely, however, requires ruling out *every* failure, including confident, well-grounded reasoning errors that leave no such trace.

6.2. RQ2: do internals add to a fair observable baseline?

Finding. In path planning, adding internals to observables lets the agent skip more simulations even under a flexible probe, and +11 to +37 points more once the baseline no longer runs part of the validator. In the CSTR, a flexible surrogate of the plant absorbs everything the internals add.

Whether internals "add" depends on the baseline they are added to. A fair comparison must rule out two objections. The first is that a linear probe handicaps the low-dimensional observable features. The second is that the path-planning observables include a shortest-path length (a breadth-first search on the prompt's graph) and an endpoint check, which are parts of

the validator's own computation. Figure 7 therefore reports the change in calls skipped when internals and grounding are added to four baselines of increasing fairness. Rows two to four come from a pre-registered sensitivity analysis. It uses a fixed, untuned gradient-boosting probe and the same certification sets, which had already been examined once, so these rows are robustness checks rather than independent confirmation. Re-fitting with the original linear probe reproduced all 60 published certification values exactly.

(a) Path planning

	Qwen2.5-1.5B	Qwen2.5-3B	Qwen2.5-7B	Llama-3.2-3B	SmolLM2-1.7B
full observables, linear probe	−0.4 [−2.0, +1.3]	**+14.0** [+12.4, +15.5]	−0.5 [−1.6, +0.6]	**+15.0** [+13.2, +16.9]	**+6.2** [+4.8, +7.5]
full observables, flexible probe	**+3.0** [+1.3, +4.9]	**+5.8** [+4.3, +7.3]	**−2.5** [−3.7, −1.3]	**+15.9** [+14.1, +17.8]	**+2.4** [+1.4, +3.4]
validator-free observables, linear probe	−1.3 [−2.8, +0.3]	**+6.9** [+5.4, +8.4]	**+1.1** [+0.2, +2.1]	**+11.6** [+9.7, +13.5]	**+9.8** [+8.4, +11.3]
validator-free observables, flexible probe	**+10.6** [+9.0, +12.3]	**+15.1** [+13.7, +16.5]	**+16.0** [+14.8, +17.5]	**+36.6** [+34.8, +38.4]	**+14.6** [+13.0, +16.2]

(b) CSTR fault recovery

	Qwen2.5-1.5B	Qwen2.5-3B†	Qwen2.5-7B	Llama-3.2-3B	SmolLM2-1.7B
full observables, linear probe	−2.2 [−5.5, +1.0]	**+8.6** [+5.0, +12.3]	**−5.6** [−8.6, −2.6]	+3.4 [−0.4, +7.2]	**+4.6** [+1.2, +8.3]
full observables, flexible probe	−0.8 [−3.9, +2.4]	**−7.8** [−11.0, −4.8]	**−3.2** [−6.0, −0.2]	**−6.1** [−9.1, −3.4]	−1.8 [−5.2, +1.6]

internals add significantly · internals cost significantly · not significant

Figure 7: When do internals add to observables? Change in the share of simulations skipped (percentage points, $X = 10\%$, paired bootstrap 95% CI) when internals and grounding are added to an observable screen. Rows are baselines of increasing fairness: linear or flexible (gradient-boosting) probe, and full or validator-free observables (without the shortest-path length and endpoint check, which reuse part of the validator). Coloured cells are significant. The first row is the confirmatory analysis; the others are a pre-registered sensitivity analysis (Table A.8). †Exploratory.

Path planning. Against the confirmatory linear baseline, internals added 14.0, 15.0 and 6.2 points for Qwen2.5-3B, Llama-3.2-3B and SmolLM2. Grounding alone added 5.1 points for Qwen2.5-1.5B and 6.5 for Qwen2.5-7B (Table A.6). Giving every signal set a flexible probe strengthened the observable screen by up to 9.4 points. The internals still added significantly for four models (+2.4 to +15.9), and Qwen2.5-7B reversed (−2.5). The decisive row is the last one. Without the two validator-derived features, a flexible observable screen skipped only 9–72% of calls, and adding internals raised that by +10.6 to +36.6 points for every model. Under the flexible probe, the internals alone skipped more calls than these validator-free observables for all five models: for example 46% against 13% for Llama-3.2-3B, and 47% against 32% for Qwen2.5-1.5B (Table A.8). This last comparison was not pre-registered and is descriptive. Put plainly, the observables were competitive in path planning largely because they ran part of the validator.

CSTR. With the linear probe, internals added for SmolLM2 (+4.6) and, in the exploratory analysis, for Qwen2.5-3B (+8.6), and reduced the calls skipped for Qwen2.5-7B (−5.6). A flexible probe made the observable screen markedly stronger, by up to 40 points, and every gain from internals disappeared. Three models skipped significantly *fewer* calls when internals were added (−3.2 to −7.8). The observable CSTR screen is a learned surrogate of the plant's response to a change from a state. Given enough flexibility, it captures what the internals contributed. Two cautions apply. No CSTR screen could be certified, because the 500-episode certification partition limits the size of the accept set (Eq. (5)); the largest accept set held 135 proposals. And the flexible observable screens of two models accepted proposals

that failed at 11.3–12.6%, above the tolerance, which the certificate would have refused. The confirmatory linear observable screens of the same models showed 10.9–11.6% and were refused as well. The safeguard therefore did its job in both cases.

Stability across development splits. The confidence intervals above capture sampling of the certification set, not the choice of the development split on which each probe was fitted and thresholded. We therefore re-partitioned the open data at random ten times, refitted and rethresholded every screen, and evaluated each split on the same certification set (Table A.9). The share of calls skipped varied with an SD of 1–7 points. In path planning, adding internals to observables raised the mean share for all five models. The gain was largest for Llama-3.2-3B (57.4% against 40.9%) and Qwen2.5-3B (37.1% against 21.9%). The internals-only screen certified in 8 of 10 splits for Qwen2.5-7B and Llama-3.2-3B, against 1 and 2 of 10 for their observable screens. In the CSTR, the differences between signal sets were mostly within the split-to-split variation. The exception was Llama-3.2-3B, for which the combined screen skipped more calls than the observable screen on average (78.6% against 68.7%). No CSTR screen certified on any split, which is the power limit again.

6.3. RQ3: why do internals help in one domain and not the other?

Finding. Low attention on the prompt region that decides an answer predicts failure when the agent's errors are failures to consult that region (path planning), but not when the agent consults the right region and reasons incorrectly (CSTR).

Region-based grounding (Eq. 9, Fig. 3) tests the mechanism directly. If a domain's failures are retrieval failures, low attention on the deciding region

should predict them; if they are reasoning failures, it should not. The contrast between the domains was written into the CSTR grounding pre-registration before any CSTR grounding value was computed.

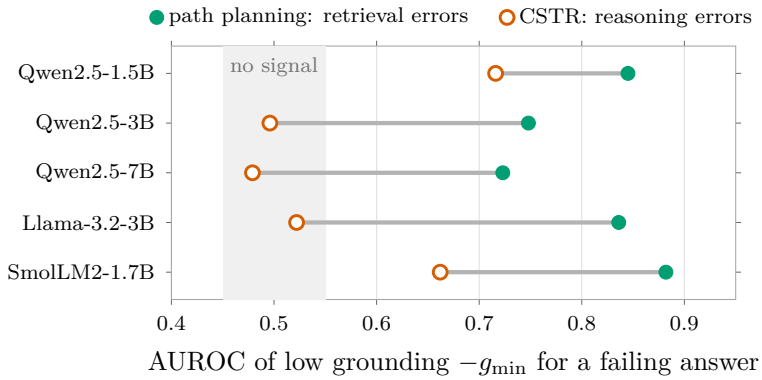


Figure 8: Does low attention on the deciding prompt region predict a failing answer? AUROC of the negative minimum grounding share $-g_{\min}$ on training and development data; the shaded band marks chance. Each line joins the same model's two domains. In path planning (filled), where errors are retrieval errors, grounding is predictive for every model. In the CSTR (open), where models consult the relevant physics and reason incorrectly, it is weak or absent. The CSTR Qwen2.5-3B value comes from the exploratory freeze.

Path planning. For every model, low grounding predicted an invalid answer (AUROC 0.72–0.88, Fig. 8). The effect is visible step by step. An illegal step received markedly less attention on its source node's adjacency line than a legal one (step-level AUROC 0.64–0.81 on development data, 0.67–0.81 on test data, four models). For Llama-3.2-3B, the deciding line was the most-attended line for 51% of legal steps but only 20% of illegal ones. As a feature, grounding raised the observable probe's AUROC by 0.03–0.05 on the test partitions (four models, all intervals above zero).

CSTR. The same signal was absent for three models (0.48–0.52) and moderate only for the two smallest (0.66 and 0.72). A pre-registered test for Qwen2.5-3B makes the difference concrete. 339 of 3954 development proposals raised the feed while cooling was saturated and the reactor was hot, a move the stated physics rules out; only 5.9% of them passed. The hypothesis was that such proposals attend *less* to the physics section. They attended slightly *more*: the AUROC in the predicted direction was 0.31. The model reads the relevant physics and still moves the feed the wrong way. A pre-registered check with three alternative region definitions (Section 8) confirmed the absence of signal for Qwen2.5-3B and Llama-3.2-3B. For Qwen2.5-7B, however, low attention on the whole physics section did predict failure (AUROC 0.70), so for that model the contrast is one of degree.

Answer to RQ3. This explains both RQ1 and RQ2. Where errors are retrieval errors, the forward pass records them and the internals beat any observable baseline that does not itself compute part of the answer. Where the outcome is a function of the plant state and the action, a surrogate of the plant captures it, and the internals' remaining contribution is to flag failures that leave a trace. Attention serves here as a predictive signal, not as an explanation of the model's computation [35, 36]. For a practitioner, the grounding AUROC on development data is a cheap test of whether internal signals are worth the extra feature pass.

6.4. Robustness to plant and fault shift

In a pre-registered shift analysis (Table A.12, Appendix A), probes were fitted on one part of the CSTR plant and fault space and evaluated with-

out recalibration on another: low to high nominal feed rate, low to high heat-transfer coefficient, and each fault family held out in turn. All signals degraded under the feed and fault-family shifts, the observable screen by 0.04–0.27 AUROC, while the cooling shift was nearly harmless. Contrary to our expectation, internal signals did not degrade less than observables. An earlier, larger shift under a different prompt version had favoured them. We report both and conclude that internal signals offer no special robustness to plant shift. Every screen must be recalibrated and re-certified when the plant or fault population changes.

7. Closed-loop results: is reading the agent safe, and when does it pay?

Offline evaluation scores one proposal per episode. A deployed screen acts inside the propose–validate–retry loop. There its rejections create new proposals, its decisions change which feedback the agent sees, and the cost of a skipped call depends on what replaces it. This section answers RQ4 on 400 fresh CSTR episodes per model (100 per fault family), disjoint from all data used to build the screens.

7.1. Policies and protocol

Every policy was run on the same 400 episodes with the frozen probes of each model. The policies are listed below.

- *Always validate* (AV), the conventional loop and the reference.
- *Never validate*, which executes the first proposal.

- *Random routing*, which accepts, rejects or validates at random with the combined probe's development rates (a control for the skip rate itself).
- *Observables*, the observable screen.
- *Combined*, the screen on observables, all internals and grounding.
- *Internals only*, the screen on all internals and grounding, without plant readings or proposed setpoints.

The probe-based policies were also run under the first-proposal-only rule (R0) wherever R0 could change the outcome (below). An unchecked rejection returns a fixed message asking for a different set of setpoints, which contains no measured metric. A validated failure returns the validator's measured outcome. A proposal that repeats an earlier one keeps its earlier verdict and costs no new call. Executing a proposal means applying its setpoints to the plant from the snapshot, with the plant's own future disturbances. The executed outcome is the simulator's verdict on that trajectory, so the validator doubles as the plant. Its decision-time calls are charged as compute, while the executed trajectory is not. Table A.10 in Appendix A gives every number with its confidence interval.

7.2. Safety and recovery

Finding. With the first-proposal-only rule, the internals-only screen, which has no plant model, executed 0–2 failing actions unchecked in 400 episodes per model while avoiding 29–65% of simulator calls. Random skipping at the same rates executed up to 186. In a pre-registered replication on 400 new episodes per model it executed none. The price is paid in recovery, and only by agents that learn from the validator's feedback.

Figure 9 shows the three quantities that matter for each policy and model. The first is the number of failing actions executed without validation. The second is the share of validator calls avoided. The third is the change in recovered episodes relative to always-validate. A good policy sits at the left of the first panel, the right of the second and close to zero in the third.

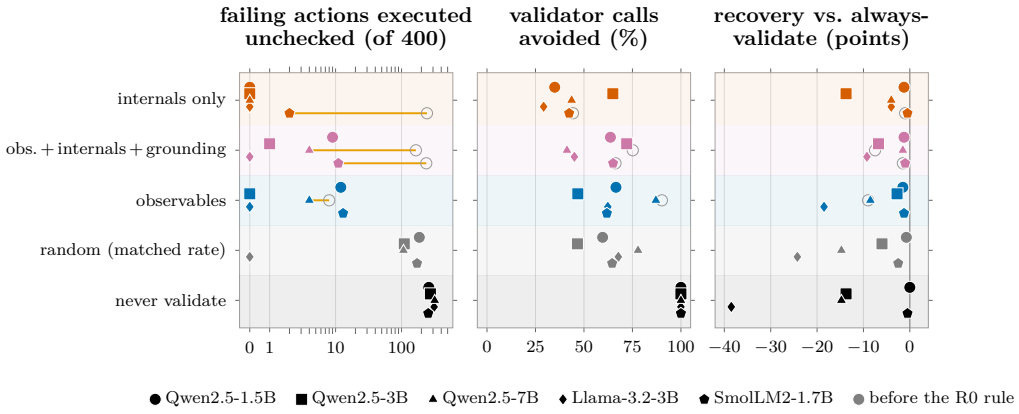


Figure 9: Closed loop on 400 fresh CSTR episodes per model. Each row is a policy; each marker is one model. Probe-based policies are shown with the first-proposal-only rule (R0) where it was run. Faded rings show the same screen before R0; for the failing-action counts, a line joins them. Left: failing actions executed without validation (log scale; 0 plotted at the left edge). Centre: validator calls avoided. Right: change in recovered episodes relative to always-validate, which recovers 34–54%.

Trained screens versus skipping at random. Never validating executed a failing action in 64–80% of episodes. Random routing at the screens' rates executed 107–186 failing actions per 400 episodes. The exception was Llama-3.2-3B, whose development accept rate, and hence the random policy's, was zero. The trained screens avoided as many calls with one to two orders of

magnitude fewer failures. Their safety therefore comes from *which* proposals they skip, not from how many.

A failure mode: internals-based probes on retries. Two models showed a failure that offline evaluation could not reveal (the faded rings in Fig. 9). Before the R0 rule, the combined screen of Qwen2.5-7B executed 165 failing actions, 161 of them on retries. SmolLM2's combined and internals-only screens executed 237 and 243, nearly all on retries. Remark 1 explains this. The probes were fitted and certified on first proposals, and a retry prompt contains feedback that a first prompt never contains. The internal state that the probe reads therefore shifts after the model has been told it failed, and the probe misreads failing retries as safe. Observable features of the plant and the proposed change do not depend on the feedback in this way. The observable screens of the same models executed 8 and 13 failing actions, and only 4 of Qwen2.5-7B's 8 were on retries.

The first-proposal-only rule restores the guarantee's premise. Under R0, a retry that the probe would accept is validated instead (Algorithm 1), so every unchecked execution is a first proposal, the population that Proposition 1 covers. The rule was pre-registered after the failure was observed and before any run with it. It reduced the failing executions from 165 to 4 (Qwen2.5-7B, combined), from 237 to 11 (SmolLM2, combined) and from 243 to 2 (SmolLM2, internals only). Recovery stayed within 1.5 points of always-validate, and 41–65% of calls were still avoided. For the other three models R0 does not change safety: in 400 episodes their probes accepted only five retries unchecked, all of which passed. With R0, the internals-only screen

executed 0, 0, 0, 0 and 2 failing actions in the 2000 episodes of the five models.

Replication on new episodes. Because R0 was designed after seeing the episodes above, those episodes cannot test it. We therefore built 400 new episodes per model (new seed, same generator) after the rule was fixed and pre-registered two criteria: the one-sided 95% upper bound on the share of episodes with an unchecked failing action must not exceed 5% (P2a, safety), and the lower end of the 95% interval of the recovery difference to always-validate must lie above -5 points (P2b, non-inferiority). Both models that had shown the retry failure were re-run with all three screens, with and without R0 (Table 5).

- *The failure replicated.* Without R0, the combined screens executed 152 (Qwen2.5-7B) and 243 (SmolLM2) failing actions, and SmolLM2's internals-only screen 252. The failure is a property of internals-based probes on retries, not of the episodes on which it was first seen.

- *R0 passed the safety criterion for all six screens.* The internals-only screens executed no failing action in 800 episodes (upper bound 0.7% per model). The worst screen, SmolLM2 combined, executed 9 (bound 3.9%).

- *Non-inferiority of recovery held for SmolLM2 and failed for Qwen2.5-7B.* SmolLM2 lost at most 1.2 points with every screen. Qwen2.5-7B lost 3.5 points with the combined and internals-only screens, with interval lower ends of -6.2 and -5.8, and 13.0 points with the observable

Table 5: Pre-registered tests of the first-proposal-only rule on 400 new CSTR episodes per model, built after the rule was fixed. R0: retries are always validated; unchecked rejections are discarded (item 2). R0+V: as R0, but rejected proposals are validated, so the agent keeps the validator's feedback (item 2b). Unsafe: failing actions executed without validation, with the one-sided 95% Clopper–Pearson upper bound on their share (criteria P2a, P2d: $\leq 5\%$). Before R0: the same screen without the rule. Δ recovered: paired difference to always-validate (points, bootstrap 95% CI; criteria P2b, P2c: lower end > -5). Saving: compute saved relative to always-validate at 60 s per validator call (Eq. 6).

Model	Screen	Rule	Unsafe	Bound (%)	Before R0	Δ recovered	Calls avoided	Saving
Qwen-7B	Observables	R0	0	0.7	–	-13.0 [-16.8, -9.2]	88%	62%
		R0+V	0	0.7		+0.0 [+0.0, +0.0]	5%	4%
	Obs.+int.+grd.	R0	0	0.7	152	-3.5 [-6.2, -1.0]	43%	26%
		R0+V	0	0.7		+0.0 [+0.0, +0.0]	3%	0%
	Internals only	R0	0	0.7	0	-3.5 [-5.8, -1.5]	46%	29%
		R0+V	0	0.7		+0.0 [+0.0, +0.0]	0%	-3%
SmolLM-1.7B	Observables	R0	8	3.6	–	-1.2 [-2.5, -0.2]	60%	38%
		R0+V	8	3.6		+0.0 [+0.0, +0.0]	23%	22%
	Obs.+int.+grd.	R0	9	3.9	243	-1.0 [-2.0, -0.2]	66%	44%
		R0+V	9	3.9		+0.0 [+0.0, +0.0]	21%	18%
	Internals only	R0	0	0.7	252	-0.8 [-1.8, +0.0]	43%	27%
		R0+V	0	0.7		+0.0 [+0.0, +0.0]	8%	8%

screen. The original episodes had suggested a loss within 1.5 points for its combined screen; the new estimate is larger and is the one we report.

Where recovery is lost. The recovery cost of the screens came almost entirely from unchecked rejections, not from unsafe accepts (Fig. 9, right). Each rejection consumes one of the six proposals and returns feedback without measurements, and some rejected proposals were good. The replication shows which of these matters. For Qwen2.5-7B, every one of the episodes that always-validate recovered and a screen lost contained an unchecked rejection, yet the rejected proposal was itself good in only 3 of 21 such episodes (combined screen) and 8 of 16 (internals only). In most of them, 18 of 21 for the combined screen, always-validate had needed a failed validation before it recovered: the agent repaired its proposal *from the validator's measured*

outcome. Qwen2.5-7B recovered 62 of 400 episodes this way, SmolLM2 one. A screen can replace the validator's verdict but not its feedback. Agents that cannot use feedback lose almost nothing to rejection; agents that can lose the episodes they would have repaired. Over the original episodes, the cost was within 1.5 points of always-validate for every screen of Qwen2.5-1.5B and SmolLM2 and for the combined R0 screen of Qwen2.5-7B. It was larger for the Qwen2.5-7B observable screen (-8.5), the Qwen2.5-3B combined and internals-only screens (-6.8 and -13.8; the latter rejected 0.45 good proposals per episode), and Llama-3.2-3B. Llama's proposals rarely pass and its screens accept nothing, so every skipped call was a rejection; its recovery fell by 4.0 (internals only) to 18.5 (observables) points. For the two smallest models validation did not improve recovery at all: never-validate recovered as often as always-validate, because these models mostly repeat their proposals. For them the validator, and the screens, serve only to prevent the execution of failing actions.

Validating rejections instead of discarding them. If rejections cost recovery because they withhold feedback, validating the rejected proposal should restore it. We pre-registered this variant (R0+V) after the replication and before any run, and tested it on the same 400 new episodes (Table 5, rows R0+V). Its recovery criterion (P2c) and safety criterion (P2d) held for every screen of both models. Recovery was identical to always-validate in every one of the 800 episodes, and the always-validate re-run reproduced the earlier run episode for episode. The restoration was complete, but so was the loss of savings. For Qwen2.5-7B, the screens avoided 0–5% of calls under R0+V against 43–88% under R0, and the compute saving at 60 s per call fell

from 26–62% to between −3 and +4%. For SmolLM2 the calls avoided fell from 43–66% to 8–23%. The reason is the same finding seen from the other side: in the CSTR, the screens skip the validator almost entirely by rejecting (Section 6.1), and Qwen2.5-7B's screens accepted almost no first proposal unchecked. A rejection saves a call *precisely by* withholding the feedback, so for an agent that repairs from feedback the call saved and the recovery lost are one and the same decision. Neither variant dominates. The choice between them is an engineering trade-off between the price of a simulation and the value of a recovered episode: for Qwen2.5-7B's combined screen, R0 buys 26% of the compute at 60 s per call for 3.5 points of recovery, and R0+V buys back the 3.5 points for almost all of the saving. For SmolLM2, which rarely repairs from feedback, R0 is nearly free and is the better choice.

7.3. When does reading the agent pay off?

Finding. Because the screen's cost is fixed while the simulator's grows with fidelity and horizon, screening saves compute once a simulation costs more than 3–18 s per call. At 60 s per call the internals-only screen saves 21–34% of total compute for four of five models.

The plant is paused during decisions, and both decoding and validation are deterministic. The logged decisions can therefore be re-costed exactly for any simulator cost (Section 3.5). Figure 10 shows the resulting compute saving relative to always-validate, as a function of the cost of one simulation.

At the simulator cost measured here (8–12 s per call), generation dominated the per-episode compute. The extra forward passes for the internal signals added 2–14 s per episode. Every unchecked rejection also triggers a regeneration that costs about as much as the simulation it replaces. As a result the screens saved little or cost time at that cost. The curves rise steeply

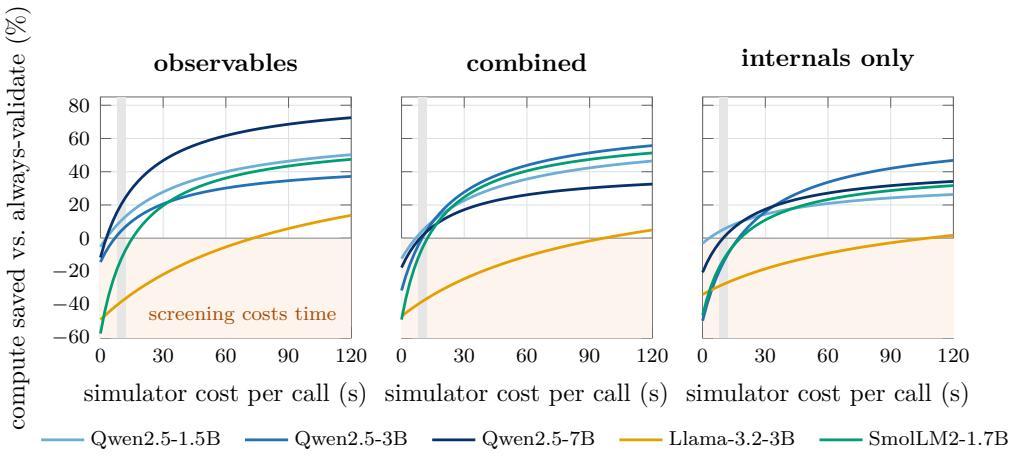


Figure 10: Compute saved relative to always-validate as the simulator becomes more expensive, from exact re-costing of the logged closed-loop decisions (Eq. 6; R0 variant where it was run). Below zero (shaded), screening costs time; the grey band marks the simulator cost measured in this study. Each curve crosses zero at the break-even cost of Eq. (7) and approaches the share of calls avoided as the simulator cost grows.

beyond it. Excluding Llama-3.2-3B, the break-even costs were 3–18 s per call (Table A.11). At 60 s per call, the internals-only screen saved 21–34% of compute, the combined screen 26–44% and the observable screen 30–62%. At 120 s the internals-only screen saved 26–47%. Llama-3.2-3B is screened only by rejection and generates slowly, so its break-even lies at 73–107 s, and its timings were further inflated by GPU memory pressure in our runs. The figure makes the engineering argument for selective validation explicit. The higher the fidelity and horizon of the digital twin, the more it pays to read the agent before running the simulator.

8. Discussion

8.1. What the agent's internals can and cannot replace

The results support a precise version of the claim that motivated this study. The forward pass that produces an LLM agent's proposal carries information about whether that proposal will pass simulation. The information is available before the simulator runs, costs a fixed amount, and requires no model of the plant. How much verification it can replace depends on the kind of error the agent makes. When errors are failures to consult information the agent was given, as in path planning, the internals support certified unchecked execution. They add to observables even under a flexible probe, and they beat any observable baseline that does not itself compute part of the validator's answer. When errors are reasoning errors over correctly consulted information, as in CSTR fault recovery, the internals still identify many failing proposals and reject them with 95–98% precision. They cannot, however, certify a proposal as safe, and a flexible surrogate of the plant's

response captures what they add. Region-based grounding makes this distinction measurable. Its AUROC on development data is a cheap diagnostic of whether internal signals are worth the extra feature pass for a given task and model.

The asymmetry between rejection and acceptance has a practical reading. A rejection by an internals-only screen is cheap to trust: it can at worst cost a regeneration, never an unsafe action. It therefore requires no plant model and no certification for safety. An acceptance is where the guarantee is needed. Our results suggest that, in physics-heavy tasks, acceptance should rely on features that describe the plant's response, while the agent's internals contribute mainly by filtering out proposals that are clearly wrong.

8.2. Plant surrogates versus model internals

A natural objection is that a screen on plant measurements and proposed setpoints is a learned surrogate of the digital twin rather than a statement about the agent. The sensitivity analysis confirms this reading: given a flexible probe, the CSTR observable screen improved by up to 40 points and absorbed every gain from internals (Fig. 7). Where a cheap surrogate of the plant can be learned, it is the better tool for *accepting* proposals. The study shows what the internals offer beyond it in three ways. First, the plant-readings-only baseline, which sees the plant state but not the proposed action, skipped nearly as many calls as the full observable screen for three models, but almost none for Qwen2.5-3B (1.8%) and Llama-3.2-3B (4.0%). For those two models the outcome depends on the agent's action, not only on the fault state. Second, the internals-only screen uses no plant information at all. With the first-proposal-only rule it avoided 29–65% of validator calls

while executing at most two failing actions in 2000 closed-loop episodes. Third, a surrogate must be retrained when the plant changes, whereas the internal signals are tied to the agent. Our shift analysis shows, however, that this does not make them more robust to a change of plant population. Both kinds of screen must be recalibrated (Section 6.4).

8.3. A deployment guide

Figure 11 condenses the results into a procedure for engineers who deploy an LLM agent behind a simulator. Three design choices mattered more than the choice of signal.

1. *Thresholds need a confidence bound.* A point-estimate accept threshold overshot the tolerance on certification data. Even with the bound-based rule, two observable CSTR screens exceeded the tolerance on new data, and the certificate caught them. Certification is therefore not a formality but the step that detects optimistic thresholds.
2. *Retries are a different population.* A screen certified on first proposals must not accept retries unchecked, because feedback changes the agent's internal state. The first-proposal-only rule turns this observation into a one-line safeguard. It removed 95–100% of the unsafe executions of the affected screens, on the episodes that revealed the failure and again on new ones.
3. *A rejection withholds the validator's feedback.* The validator returns not only a verdict but a measured outcome, and an agent that can use it repairs its next proposal. An unchecked rejection replaces that outcome with a bare request to try again. For SmolLM2, which almost

never repaired from feedback, this cost at most 1.2 points of recovery; for Qwen2.5-7B, which recovered one episode in six only after a failed validation, it cost 3.5 points and failed the pre-registered non-inferiority margin. Validating rejected proposals restored recovery exactly, but in the CSTR nearly all of the saving came from rejections, so it also removed nearly all of the saving. A rejection saves a call by withholding the feedback; for an agent that repairs from feedback, the two cannot be separated. The screen's operator must therefore price both: the cost of a simulation and the value of a recovered episode. A screen whose savings come from certified *accepts*, as in path planning, does not face this trade-off, which makes accept-side certification the more valuable target for capable agents.

4. *Savings depend on what replaces a skipped call.* An unchecked accept saves a full validation. An unchecked rejection of a good proposal forces a regeneration. Screening therefore pays when a validation costs more than the break-even c^*, which was 3–18 s for four of five models. This is the regime that motivates selective validation in the first place: high-fidelity or long-horizon simulators, scenario sets and multi-unit plants, where the cost of validation grows while the cost of the agent stays fixed.

8.4. Threats to validity

Internal validity. All hypotheses, rules and amendments were pre-registered, and sealed partitions were evaluated once. Two analyses are exploratory and labelled as such: the offline CSTR result of Qwen2.5-3B and the closed-loop probes of that model. The first-proposal-only rule was introduced after

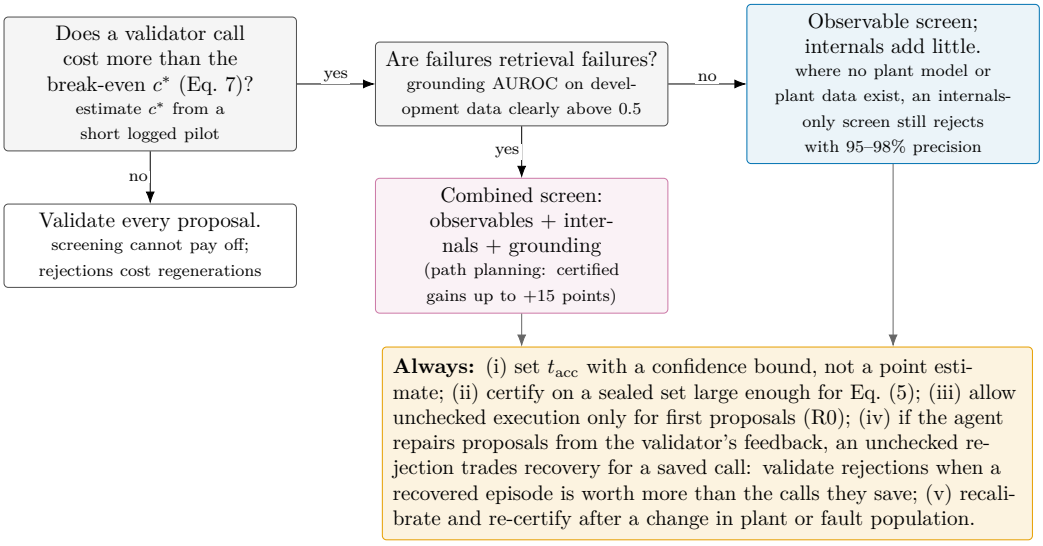


Figure 11: Deployment guide derived from the results. The first question decides whether screening can pay off (Section 7). The second decides whether internal signals are worth adding to an observable screen (Section 6.3). The rules at the bottom apply to every screen.

observing the retry failure. We therefore re-tested it, under pre-registered criteria, on 400 new episodes per affected model: its safety criterion held for all six screens, its recovery criterion only for SmolLM2 (Section 7). Teacher-forced re-runs agreed with greedy generation for 92.8–99.9% of answer tokens for the bfloat16 models but only 84–91% for the 4-bit Qwen2.5-7B. For that model the internal features describe a closely related, not identical, decoding path. This may contribute to its weaker gains from internals.

Construct validity. The CSTR deciding regions are domain-informed choices taken from the process structure stated in the prompt. In a pre-registered sensitivity check we recomputed the CSTR grounding AUROC under three alternative region definitions: the physics section, the section describing how each setpoint acts, and the whole snapshot. For Qwen2.5-3B and Llama-3.2-3B no definition reached 0.55. For Llama, *more* attention on the physics section went with failure (0.35), as in the pre-registered H3 test. For Qwen2.5-7B, however, low attention on the physics and action sections did predict failure (0.70 and 0.71), which our pre-registered criterion had not anticipated. For that model the CSTR failures are partly failures to consult the physics, and the retrieval–reasoning contrast is one of degree rather than kind. Attention is used as a predictive signal, not as an explanation. The observable baseline in path planning includes a shortest-path length and an endpoint check, which are partial validator computations. This makes the baseline strong and the gains from internals conservative.

External validity. The study uses open-weight models of 1.5–7 billion parameters, one of them quantised, on a single laptop GPU. Larger models make fewer retrieval errors and may therefore benefit less from grounding,

and API-served models do not expose internal signals at all. The CSTR is one simulated plant with four fault families and one prompt, and the plant is paused while the agent and the validator run. We make no claim about decision latency under real-time constraints. The validator doubles as the plant in closed loop, so model mismatch between the digital twin and a real plant is outside the scope of this study.

Statistical conclusion validity. Certification in the CSTR was limited by the size of the certification partitions (Eq. 5), so the absence of certified CSTR screens reflects power as well as risk. Confidence intervals are per comparison. The certification bounds are Bonferroni-corrected over signal sets but not over models or tolerances. Compute figures are specific to our hardware, but the linear re-costing makes their dependence on the validator explicit and transferable.

9. Conclusion

LLM agents for process operations need not pay for a full simulation of every proposal, because the forward pass that produces a proposal already estimates how likely it is to fail. Across two domains, five open-weight models and a pre-registered protocol with sealed test sets, a screen that reads the agent's internals, with no model of the plant, removed 11–79% of validator calls in graph path planning, with certified accepts for four models, and 12–53% in CSTR fault recovery, almost entirely through rejections that were 95–98% precise. Where the agent's errors were failures to consult the relevant information, internals added to observable features under a flexible probe and added 11–37 points against observables that do not reuse the validator's

logic. Where the agent consulted the right information and reasoned incorrectly, a flexible surrogate of the plant absorbed their contribution. Region-based attention grounding measures this distinction directly. In closed loop, internals-based screens accepted failing retries, a distribution shift outside the certificate; allowing unchecked execution only for first proposals restored the certificate's premise and kept unsafe executions at 0–2 per 400 episodes while 29–65% of validator calls were avoided. A pre-registered replication on new episodes confirmed the safety of the rule but showed its limit: a screen can replace the simulator's verdict, not its feedback, so an agent that repairs its proposals from that feedback loses recovery to unchecked rejections. Validating the rejected proposals restored that recovery exactly but, because the reactor screens saved calls mainly by rejecting, also gave up most of the saving: for such agents the saved call and the lost recovery are the same decision, to be priced by the operator. Because the screen's cost is fixed and the simulator's grows with fidelity and horizon, these avoided calls translated into compute savings of 21–62% at 60 s per validation for four of five models.

Future work should train and certify screens on retries rather than excluding them, raise the accept-side coverage of screens for agents that learn from feedback, where an unchecked accept costs no feedback, certify CSTR screens on larger partitions, test larger and API-served models through whatever internal signals they expose, and evaluate the approach on plants with richer fault libraries, real-time constraints and model mismatch between the digital twin and the plant.

CRediT authorship contribution statement

[TODO: CRediT roles per author.]

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

[TODO: Elsevier requires this statement. Describe the use of AI assistance for code, analysis and drafting, and confirm that the authors reviewed and edited the content and take full responsibility for it.]

Data availability

Code, pre-registrations with dated amendments, frozen probes' manifests and all results tables are available at [TODO: repository URL / Zenodo DOI]. Raw model records and hidden-state arrays are available on request [TODO: or deposit on Zenodo].

Acknowledgements

[TODO: Funding and acknowledgements.]

Table A.6: Path planning: share of validator calls skipped (%) per signal on the fresh certification set (tolerance $X = 10\%$). Bold: failure rate among unchecked accepts certified $\leq 10\%$ (one-sided Clopper–Pearson, Bonferroni over signals). $^{+}/^{-}$: significantly more/fewer calls skipped than observables (paired bootstrap 95% CI).

Signal	Qwen-1.5B	Qwen-3B	Qwen-7B	Llama-3B	SmolLM-1.7B
Observables	**40**	**21**	**16**	43	85
Obs. + grounding	**46**$^{+}$	**21**	22^{+}	**59**$^{+}$	85
Obs. + all internals	**37**$^{-}$	**32**$^{+}$	15	**47**$^{+}$	79^{-}
Obs. + internals + grounding	**40**	**34**$^{+}$	**15**	**58**$^{+}$	91^{+}
Region grounding	**31**$^{-}$	**14**$^{-}$	**3**$^{-}$	**31**$^{-}$	80^{-}
Attention (regions)	**35**$^{-}$	**21**	13^{-}	17^{-}	75^{-}
Hidden states	22^{-}	**24**$^{+}$	10^{-}	27^{-}	74^{-}
All internals	**25**$^{-}$	**25**$^{+}$	**9**$^{-}$	**31**$^{-}$	76^{-}
All internals + grounding	**38**$^{-}$	**23**$^{+}$	**11**$^{-}$	**36**$^{-}$	79^{-}
Token confidence	13^{-}	18^{-}	0^{-}	14^{-}	56^{-}

Table A.7: CSTR: share of validator calls skipped (%) per signal on the certification partition ($X = 10\%$); notation as in Table A.6. †Qwen-3B: exploratory re-analysis under the bound-based rule (its sealed sets had been used once before the rule change).

Signal	Qwen-1.5B	Qwen-3B†	Qwen-7B	Llama-3B	SmolLM-1.7B
Observables	63	41	86	71	61
Obs. + grounding	60^{-}	39	86	75^{+}	65^{+}
Obs. + all internals	61	51	83	72	66^{+}
Obs. + internals + grounding	61	50	80^{-}	74	66^{+}
Region grounding	27^{-}	10	47^{-}	38^{-}	24^{-}
Attention (regions)	30^{-}	15	49^{-}	48^{-}	31^{-}
Hidden states	29^{-}	13	50^{-}	48^{-}	45^{-}
All internals	29^{-}	15	47^{-}	53^{-}	40^{-}
All internals + grounding	28^{-}	12	48^{-}	53^{-}	41^{-}
Token confidence	4^{-}	0	17^{-}	34^{-}	22^{-}

Appendix A. Complete results tables

This appendix lists every number behind the figures of Sections 6 and 7. All tables are generated from the saved results by the scripts listed in the data availability statement.

Table A.8: Sensitivity analysis (pre-registered, on previously examined certification sets): share of validator calls skipped (%, $X = 10\%$) with the confirmatory linear probe and a fixed, untuned gradient-boosting probe. Validator-free observables omit the shortest-path length and the endpoint check. Bold: certified (Bonferroni over the screens of this analysis). The CSTR Qwen-3B column is exploratory.

Domain	Signal	Probe	Qwen-1.5B	Qwen-3B	Qwen-7B	Llama-3B	SmolLM-1.7B
Path planning	observables	linear	**40.4**	**20.5**	**15.7**	43.0	84.6
		boosting	**44.8**	**27.1**	**15.8**	49.5	**94.0**
	validator-free observables	linear	**33.4**	**18.2**	**11.7**	22.0	71.1
		boosting	32.3	**12.5**	**8.9**	13.2	72.4
	internals only	linear	**38.0**	**22.6**	**10.8**	**35.8**	78.8
		boosting	**47.1**	30.0	**11.9**	**45.9**	86.7
	combined	linear	**39.9**	**34.5**	**15.2**	**58.1**	90.7
		boosting	**47.8**	**32.9**	**13.3**	**65.5**	96.3
	validator-free combined	linear	**32.1**	**25.1**	**12.8**	**33.6**	80.9
		boosting	**42.9**	27.5	**25.0**	**49.8**	87.0
CSTR	observables	linear	62.8	41.2	85.5	70.7	61.1
		boosting	71.7	81.5	87.3	89.5	69.4
	internals only	linear	28.3	12.2	47.8	52.7	40.5
		boosting	27.6	22.5	49.4	54.2	43.5
	combined	linear	60.6	49.8	79.9	74.1	65.7
		boosting	70.9	73.7	84.1	83.3	67.5

Table A.9: Stability across development splits: share of validator calls skipped (%, $X = 10\%$) over 10 random re-partitions of the open data (stratified; probes refitted and rethresholded each time), evaluated on the same certification sets. Mean ± SD [min–max]; c: splits in which the screen is certified. The CSTR Qwen-3B column is exploratory.

Domain	Signal	Qwen-1.5B	Qwen-3B	Qwen-7B	Llama-3B	SmolLM-1.7B
Path planning	observables	41.6 ± 4.2 [37–51], c=9	21.9 ± 1.2 [20–24], c=9	15.6 ± 3.4 [7–21], c=1	40.9 ± 3.7 [35–46], c=2	82.6 ± 1.2 [81–85], c=3
	internals only	43.5 ± 3.9 [38–49], c=5	34.1 ± 7.0 [25–46], c=8	14.4 ± 4.0 [7–19], c=8	35.3 ± 5.6 [26–43], c=8	76.7 ± 3.0 [72–81], c=0
	combined	48.6 ± 4.6 [43–59], c=9	37.1 ± 5.1 [31–46], c=7	18.1 ± 3.4 [10–22], c=7	57.4 ± 2.8 [53–61], c=5	87.9 ± 1.9 [85–91], c=4
CSTR	observables	65.9 ± 6.5 [53–73], c=0	46.1 ± 5.1 [42–56], c=0	87.3 ± 2.0 [85–90], c=0	68.7 ± 3.8 [64–76], c=0	65.5 ± 6.8 [52–72], c=0
	internals only	29.3 ± 1.2 [28–32], c=0	16.1 ± 4.3 [9–22], c=0	43.5 ± 7.4 [27–51], c=0	59.3 ± 4.7 [51–67], c=0	43.0 ± 5.4 [31–49], c=0
	combined	62.7 ± 4.8 [55–69], c=0	45.5 ± 3.9 [38–50], c=0	80.7 ± 5.3 [70–88], c=0	78.6 ± 1.7 [77–82], c=0	62.4 ± 5.2 [51–67], c=0

Table A.10: Closed-loop CSTR recovery on 400 fresh episodes per model. Δ: paired difference to always-validate (percentage points, bootstrap 95% CI over episodes). Unsafe: failing actions executed without validation. Calls: validator calls avoided relative to always-validate. R0: the probe may skip validation only for first proposals (retries are always validated).

Model	Policy	Recovered (%)	Δ recovered	Unsafe	Calls avoided
Qwen-1.5B	Always validate	34.5	–	0	–
	Observables	33.0	-1.5 [-3.0, -0.2]	12	67%
	Obs.+int.+grd.	33.2	-1.2 [-2.5, +0.0]	9	64%
	Internals only	33.2	-1.2 [-2.5, -0.2]	0	35%
	Random (matched)	33.8	-0.8 [-1.8, +0.0]	186	60%
	Never validate	34.5	+0.0 [+0.0, +0.0]	259	100%
Qwen-3B	Always validate	43.8	–	0	–
	Observables	41.0	-2.8 [-5.8, +0.2]	0	47%
	Obs.+int.+grd.	37.0	-6.8 [-10.2, -3.5]	1	72%
	Internals only	30.0	-13.8 [-17.5, -10.5]	0	65%
	Random (matched)	37.8	-6.0 [-9.0, -3.0]	110	47%
	Never validate	30.0	-13.8 [-17.2, -10.5]	274	100%
Qwen-7B	Always validate	34.0	–	0	–
	Observables	25.0	-9.0 [-12.8, -5.2]	8	90%
	Observables, R0	25.5	-8.5 [-12.2, -4.8]	4	87%
	Obs.+int.+grd.	26.5	-7.5 [-11.0, -4.0]	165	75%
	Obs.+int.+grd., R0	32.5	-1.5 [-4.2, +1.2]	4	41%
	Internals only	30.0	-4.0 [-6.2, -1.8]	0	44%
	Internals only, R0	30.0	-4.0 [-6.2, -1.8]	0	44%
	Random (matched)	19.2	-14.8 [-18.5, -10.8]	107	78%
	Never validate	19.2	-14.8 [-18.5, -11.2]	318	100%
Llama-3B	Always validate	53.8	–	0	–
	Observables	35.2	-18.5 [-24.2, -13.2]	0	62%
	Obs.+int.+grd.	44.5	-9.2 [-14.2, -4.8]	0	45%
	Internals only	49.8	-4.0 [-8.0, +0.0]	0	29%
	Random (matched)	29.5	-24.2 [-29.5, -19.5]	0	68%
	Never validate	15.2	-38.5 [-43.5, -33.8]	312	100%
SmolLM-1.7B	Always validate	36.5	–	0	–
	Observables	35.2	-1.2 [-2.2, -0.2]	13	62%
	Observables, R0	35.2	-1.2 [-2.2, -0.2]	13	62%
	Obs.+int.+grd.	35.0	-1.5 [-2.8, -0.5]	237	66%
	Obs.+int.+grd., R0	35.5	-1.0 [-2.0, -0.2]	11	65%
	Internals only	35.5	-1.0 [-2.0, -0.2]	243	44%
	Internals only, R0	36.0	-0.5 [-1.2, +0.0]	2	42%
	Random (matched)	34.0	-2.5 [-4.2, -1.0]	172	65%
	Never validate	36.0	-0.5 [-1.2, +0.0]	254	100%

Table A.11: Compute per episode relative to always-validate (positive = saving). The logged closed-loop decisions do not depend on validator latency (the plant is paused during decisions), so the same decisions are re-costed exactly for a validator taking 30 s or 60 s per call. Break-even: validator cost per call above which the policy saves time.

Model	Policy	Measured	30 s	60 s	Break-even (s)
Qwen-1.5B	Observables probe	+11%	+28%	+40%	3
	Obs. + internals + grounding probe	+5%	+23%	+36%	7
	Internals-only probe	+6%	+14%	+21%	3
Qwen-3B	Observables probe	+2%	+21%	+30%	7
	Obs. + internals + grounding probe	-4%	+28%	+44%	10
	Internals-only probe	-19%	+16%	+34%	17
Llama-3B	Observables probe	-40%	-22%	-5%	73
	Obs. + internals + grounding probe	-40%	-24%	-11%	97
	Internals-only probe	-29%	-18%	-9%	107
Qwen-7B	Observables probe	+18%	+49%	+64%	2
	Observables probe, first-proposal-only	+17%	+47%	+62%	3
	Obs. + internals + grounding probe	+31%	+50%	+59%	any
	Obs. + internals + grounding, first-proposal-only	-1%	+17%	+26%	9
	Internals-only probe	-1%	+18%	+28%	9
	Internals-only, first-proposal-only	-2%	+17%	+27%	10
SmolLM-1.7B	Observables probe	+0%	+24%	+39%	12
	Observables probe, first-proposal-only	-7%	+20%	+36%	15
	Obs. + internals + grounding probe	+44%	+52%	+58%	any
	Obs. + internals + grounding, first-proposal-only	-1%	+25%	+41%	12
	Internals-only probe	+40%	+42%	+43%	any
	Internals-only, first-proposal-only	-9%	+11%	+23%	18

Table A.12: AUROC for failure on the target part of each pre-registered shift (probes fitted on the source part only). Control: random split of the same size.

Model	Signal	Control	Feed	Cooling	Fault family
Qwen-1.5B	Observables	0.93	0.70	0.91	0.72
	Obs. + all internals	0.92	0.72	0.90	0.74
	All internals	0.80	0.64	0.79	0.66
	Token confidence	0.65	0.54	0.63	0.50
Qwen-3B	Observables	0.87	0.73	0.86	0.80
	Obs. + all internals	0.85	0.74	0.85	0.74
	All internals	0.71	0.60	0.72	0.62
	Token confidence	0.57	0.48	0.55	0.50
Qwen-7B	Observables	0.93	0.66	0.92	0.77
	Obs. + all internals	0.92	0.65	0.91	0.71
	All internals	0.80	0.52	0.77	0.55
	Token confidence	0.68	0.47	0.66	0.54
Llama-3B	Observables	0.90	0.86	0.91	0.80
	Obs. + all internals	0.92	0.91	0.93	0.82
	All internals	0.84	0.83	0.86	0.71
	Token confidence	0.63	0.57	0.66	0.57

References

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: International Conference on Learning Representations, 2023.

[2] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao, Reflexion: Language agents with verbal reinforcement learning, in: Advances in Neural Information Processing Systems, 2023.

[3] B. Decardi-Nelson, A. S. Alshehri, A. Ajagekar, F. You, Generative AI and process systems engineering: The next frontier, Computers & Chemical Engineering 187 (2024) 108723.

[4] J. Vyas, M. Mercangöz, Autonomous industrial control using an agentic framework with large language models, arXiv preprint arXiv:2411.05904 (2024).

[5] J. Vyas, et al., From detection to action: LLM agents for fault-tolerant control, tODO: complete with the published Journal of Process Control reference (authors, volume, pages, DOI) (2026).

[6] F. Tao, H. Zhang, A. Liu, A. Y. C. Nee, Digital twin in industry: State-of-the-art, IEEE Transactions on Industrial Informatics 15 (4) (2019) 2405–2415.

[7] S. Kadavath, et al., Language models (mostly) know what they know, arXiv preprint arXiv:2207.05221 (2022).

[8] A. Azaria, T. Mitchell, The internal state of an LLM knows when it's lying, in: Findings of the Association for Computational Linguistics: EMNLP 2023, 2023.

[9] C. Burns, H. Ye, D. Klein, J. Steinhardt, Discovering latent knowledge in language models without supervision, in: International Conference on Learning Representations, 2023.

[10] S. Marks, M. Tegmark, The geometry of truth: Emergent linear structure in large language model representations of true/false datasets, arXiv preprint arXiv:2310.06824 (2023).

[11] C. Chen, K. Liu, Z. Chen, Y. Gu, Y. Wu, M. Tao, Z. Fu, J. Ye, INSIDE: LLMs' internal states retain the power of hallucination detection, in: International Conference on Learning Representations, 2024.

[12] H. Orgad, M. Toker, Z. Gekhman, R. Reichart, I. Szpektor, H. Kotek, Y. Belinkov, LLMs know more than they show: On the intrinsic representation of LLM hallucinations, in: International Conference on Learning Representations, 2025.

[13] Y.-S. Chuang, L. Qiu, C.-Y. Hsieh, R. Krishna, Y. Kim, J. Glass, Lookback lens: Detecting and mitigating contextual hallucinations in large language models using only attention maps, in: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, 2024.

[14] C. J. Clopper, E. S. Pearson, The use of confidence or fiducial limits

illustrated in the case of the binomial, Biometrika 26 (4) (1934) 404–413.

[15] Qwen Team, Qwen2.5 technical report, arXiv preprint arXiv:2412.15115 (2024).

[16] A. Grattafiori, et al., The llama 3 herd of models, arXiv preprint arXiv:2407.21783 (2024).

[17] L. Ben Allal, et al., SmolLM2: When smol goes big – data-centric training of a small language model, arXiv preprint arXiv:2502.02737 (2025).

[18] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, D. T. Mellor, The preregistration revolution, Proceedings of the National Academy of Sciences 115 (11) (2018) 2600–2606.

[19] K. P. Wabersich, M. N. Zeilinger, A predictive safety filter for learning-based control of constrained nonlinear dynamical systems, Automatica 129 (2021) 109597.

[20] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, U. Topcu, Safe reinforcement learning via shielding, in: Proceedings of the AAAI Conference on Artificial Intelligence, 2018.

[21] A. I. J. Forrester, A. J. Keane, Recent advances in surrogate-based optimization, Progress in Aerospace Sciences 45 (1–3) (2009) 50–79.

[22] Y. Geifman, R. El-Yaniv, Selective classification for deep neural networks, in: Advances in Neural Information Processing Systems, 2017.

[23] A. Kamath, R. Jia, P. Liang, Selective question answering under domain shift, in: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, 2020.

[24] D. Madras, T. Pitassi, R. Zemel, Predict responsibly: Improving fairness and accuracy by learning to defer, in: Advances in Neural Information Processing Systems, 2018.

[25] H. Mozannar, D. Sontag, Consistent estimators for learning to defer to an expert, in: Proceedings of the 37th International Conference on Machine Learning, 2020.

[26] S. Bates, A. Angelopoulos, L. Lei, J. Malik, M. I. Jordan, Distribution-free, risk-controlling prediction sets, Journal of the ACM 68 (6) (2021).

[27] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, L. Lei, Learn then test: Calibrating predictive algorithms to achieve risk control, arXiv preprint arXiv:2110.01052 (2021).

[28] A. N. Angelopoulos, S. Bates, Conformal prediction: A gentle introduction, Foundations and Trends in Machine Learning 16 (4) (2023) 494–591.

[29] M. Xiong, Z. Hu, X. Lu, Y. Li, J. Fu, J. He, B. Hooi, Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs, in: International Conference on Learning Representations, 2024.

[30] G. Alain, Y. Bengio, Understanding intermediate layers using linear classifier probes, arXiv preprint arXiv:1610.01644 (2016).

[31] Y. Belinkov, Probing classifiers: Promises, shortcomings, and advances, Computational Linguistics 48 (1) (2022) 207–219.

[32] L. Kuhn, Y. Gal, S. Farquhar, Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation, in: International Conference on Learning Representations, 2023.

[33] S. Farquhar, J. Kossen, L. Kuhn, Y. Gal, Detecting hallucinations in large language models using semantic entropy, Nature 630 (2024) 625–630.

[34] J. Kossen, J. Han, M. Razzak, L. Schut, S. Malik, Y. Gal, Semantic entropy probes: Robust and cheap hallucination detection in LLMs, arXiv preprint arXiv:2406.15927 (2024).

[35] S. Jain, B. C. Wallace, Attention is not explanation, in: Proceedings of NAACL-HLT, 2019.

[36] S. Wiegreffe, Y. Pinter, Attention is not not explanation, in: Proceedings of EMNLP-IJCNLP, 2019.

[37] J. C. Platt, Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods, in: Advances in Large Margin Classifiers, MIT Press, 1999, pp. 61–74.

[38] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer, QLoRA: Efficient finetuning of quantized LLMs, in: Advances in Neural Information Processing Systems, 2023.